

HARDWARE MODEL OF A MIXED RADIX FAST FOURIER TRANSFORM FOR LTE 3GPP

Thesis

Presented to the Faculty of the Graduate School

of Cornell University

In Partial Fulfillment of the Requirements for the Degree of

Master of Science

by

Shashank Adimulam

August 2015

© 2015 Shashank Adimulam

All Rights Reserved

Acknowledgements

I would like to sincerely thank Cornell University for letting me fulfil my dream of being a student here. I would like to thank the Electrical and Computer Engineering Department and the student services centre for giving me the opportunity to honour thesis. I am extremely grateful to my committee consisting of Prof. Christoph Studer and Prof. David Bindel for the continuous assistance and suggestions throughout my project. In addition, I would also like to thank my fellow researcher Charles Jeon, visiting researchers Florian for their continuous feedback on my work. My thesis wouldn't have been complete without support of my parents, friends and family members, thank you for always having confidence in me.

Abstract

Speeding up Discrete Fourier Transforms (DFT) computations is critical for applications in signal processing and telecommunications. The goal of this thesis is to design a hardware-aware Fast Fourier Transform (FFT) which is suitable for 3GPP - LTE applications that aims at the performance improvement and most of the work focuses on resolving memory contentions, proposing address generation schemes for mixed radix FFTs. A generalized mixed-radix algorithm was analyzed for memory based Fast Fourier Transform processors to support prime-sized and traditional 2^n point FFTs simultaneously. A Novel Memory partitioning scheme was discussed to facilitate improved speed of operation for computing FFTs. The proposed algorithm transforms single index into multi-dimensional vector for efficient computation. Existing algorithms are complex and are not hardware friendly because they do not take into account the inherent parallelism in calculating FFTs, so there is a necessity to come up with algorithms that are hardware friendly and that can account for the existing parallelism. A radix unit that computes N-point FFT needs to process N elements in parallel. Dividing single port memory into multiple banks and mapping data samples into different banks is the key to avoid contentions while reading N-elements in parallel. Novel mapping and banking schemes are discussed to avoid address conflicts while generating address to read elements simultaneously. Mapping elements from single port memory to multi-bank system uses mathematical transformations involving single dimensional index vector into multi-dimensions. These transformations ensure that the generalized mixed radix algorithm can support in place policy i.e., writing and reading from the same memory location which minimises the memory size and multi bank memory structures, increasing the maximum throughput without memory conflict. The technique that we discussed for computing FFTs leads to the design of

architectures that are parallel in nature and produce high throughput and great speeds

Biographical Sketch

Shashank Adimulam is currently pursuing his Master's in Electrical and Computer Engineering program from Cornell University. In the summer of August 2015, he will graduate with an MS degree in Electrical and Computer Engineering. He is member of Computer Systems Laboratory, where he was involved in development and the testing of an Arm core for Mobile platform on Virtex-6 FPGA. In this project, he was involved design and verification of parts of processor in Verilog. During the last summer, he worked on the development of PCB for testing the functionality of in-house fabricated Async-FPGA along with FIFO to establish a synchronous interface. His course work at Cornell include Computer Architecture, VLSI design, Parallel computing and Artificial Intelligence.

Before pursuing his MS at Cornell, he was working at Indian Space Research Organization where he worked on developing applications such as Merged Coordinate Rotation Digital Computer for geometric corrections in Satellite images on FPGA. Also, he was involved in developing Image processing applications related to Satellite data on FPGAs. His interests are developing digital logics on FPGAs and ASICs. Apart from academics, Shashank enjoys outside activities like cricket, basketball.

Declaration of Originality

I hereby confirm that I am the sole author of the entire work enclosed here, and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisors.

Shashank Adimulam,
Ithaca, July 2015

Contents

1	Introduction	1
1.1	Various FFT architectures	1
1.2	DFTs In 3GPP LTE	2
1.2.1	Receiver station for LTE	4
2	Preliminaries	8
2.1	Decimation-in-time fast fourier transform	8
2.1.1	Frequency domain transformation equation	8
2.1.2	Mathematical complexity and periodicity	9
2.1.3	Mathematics behind DIT transformation	10
2.2	Decimation-in-frequency algorithm	13
2.2.1	Mathematics behind DIF transformation	13
2.2.2	Difference between DIT and DIF	14
2.3	Generalized approach to FFT in mixed radix form	15
2.3.1	Basis behind mapping from one dimension to two dimensions	16
2.3.2	Examples with detailed analysis	17
2.3.3	Row and Column Permutations	18
2.4	30 point FFT using mixed radix	21
2.4.1	Row and column divisions	21
2.4.2	Algorithm in detail for a 30-point FFT	22
2.4.3	Radix calculations	22
2.4.4	Simplification to common index	24

Contents

2.5	Data shuffling in mixed-radix implementations	27
2.5.1	General formula	27
2.5.2	Example for index shuffling at the output	29
3	Hardware architecture for single memory mixed radix FFT and idea of banking	32
4	Discussion / Algorithms	36
4.1	Problems in implementation	37
5	Mixed Radix Design Improvements	39
5.1	In place strategy for Radix-r algorithm	40
5.2	Mixed radix FFT schemes for odd power of 2	43
6	Prime Factorisation Algorithm (PFA) for Fast Fourier Transforms	46
6.1	Evolution of PFA	46
6.2	Prime Factor Algorithm and its implications	50
6.2.1	PFA for FFT applications	53
6.2.2	More than two dimensional transformation using PFA on DFT's	58
6.2.3	Examples of PFA on DFT's	58
6.3	Banking scheme for conflict free processing	61
6.3.1	Examples of Mixed Radix FFT's using Banking schemes and PFA	63
7	Algorithm for in-place mixed radix FFT scheme using PFA	70
8	Hardware Architecture for mixed radix FFT using PFA	72
8.1	Single Port Memory for Multiple Banks	72
8.2	Improved Hardware Architecture for throughput	73
9	Speedup comparison	77
10	Conclusion and Future Work	80

List of Figures

1	Multiband of SC-FDMA transmission In LTE	3
2	Frequency to time domain transformation	4
3	Time to frequency domain transformaiton	5
4	Block lengths in LTE	6
5	Block lengths in LTE (Cont.)	7
6	Division in 8-point FFT	11
7	8-point DIT example	12
8	8-point Decimation in Frequency	15
9	30-point mixed radix example using row and column permutations	23
10	Radix-5 example	24
11	30-point FFT showing decomposition	26
12	6 point mixed radix FFT	27
13	6-point decomposition	27
14	12-point Bit reversal example	28
15	Hardware architecture for single memory mixed radix FFT	33
16	Reading issues associated with single port memories	33
17	Conflict free mapping without memory conflicts	34
18	Problem persists even after bank division	34
19	Ideal solution for multi bank Approach	35
20	Banking Scheme N = 16	42

List of Figures

21	Factors are coprimes to each other $N = 10$	64
22	Factors are NOT coprimes to each other $N = 12$	65
23	Simple three dimensional transformation $N = 12$	67
24	Three dimensional transformation with complex twiddle factors $N = 24$	69
25	Mixed radix FFT hardware model	73
26	Issues with single port memory banks	74
27	Modified Hardware model using Duplicate Banking Scheme	75
28	Pipeline diagram for Duplicate Banking scheme	75
29	A complete solution for 6 point	76
30	Speedup table for various bank lengths	78
31	Comparison of OLD and NEW mixed radix scheme	79

List of Tables

1	frequency domain arrangement	19
2	Digit-reverse in Frequency domain	30
3	Digit-reverse in Time domain	31

1 Introduction

The Discrete Fourier Transform is useful in transforming a time domain signal to frequency domain. A time varying signal is difficult to analyze due to its nature, but it is easier to analyse a signal in Frequency domain. [1] Fast Fourier Transform is an algorithm to compute the Discrete Fourier Transform. The Fast Fourier transform is one of the most prominent algorithm and is used frequently in [15] Long Term Evolution (LTE) for Third Generation Partnership Project (3GPP). LTE is commonly known as 4G LTE standard for wireless communication of high speed data for mobile phone and data terminals. All these standards are developed by 3rd Generation Partnership Project. Carriers using both GSM/UMTS networks can use the LTE as a natural upgrade path. The goal of LTE is to increase the capacity and speed of wireless data networks using the new Digital Signal Processing techniques and modulations that have been developed over the decade. One of the most important area where huge developments have been made is the improvements in the speed of computing DFTs for frequency domain analysis. In LTE system, Discrete Fourier transform (DFT)/Inverse-DFT is used and it is a very important operation. Since DFT/IDFT computation requires a large amount of arithmetic operations, we need an efficient FFT algorithm which can reduce the number of arithmetic operations to meet real time computation in LTE systems. There are many ways in which we can implement the FFT architectures.

1.1 Various FFT architectures

FFT architectures are usually divided into two main categories : The parallel architecture and the shared memory architecture. For a parallel architecture, large number of N-point FFT units

1 Introduction

operate in parallel improving the overall throughput at which the data samples are converted to frequency domain. [9] These N-point FFT units known as radix units are also known as the butterfly units. On the other hand, shared memory architecture need only few radix units to perform all the operations. Shared memory architectures compute FFT serially, but has a low throughput and requires complex circuit design of memory address controller in order to avoid memory conflicts while reading elements simultaneously for individual radix units. The biggest advantage of the shared memory architecture is the potential increase in throughput after using a higher radix algorithm. As the number of inputs to a radix unit increases, the complexity of the memory address controller also increases and there is a necessity to come up with novel architectures that are hardware friendly and can process elements simultaneously.

There are various ways of implementing FFTs and they depend strongly on the N-point FFT units which we are planning to use. For example, we can start with a simple FFT that is in the form of 2^n number of points and use radix-2, radix-4 and radix-8 unit to transform from the time domain to the frequency domain. [3] Split radix implementations, takes the advantage of both radix-2/4. Mixed radix implementations includes a prime number radix which is not a multiple of two. FFT is a faster way of implementing Discrete Fourier Transform (DFT) which transforms the time domain signal into a frequency domain, since it is often easier to analyse the signal in frequency. DFT is obtained by decomposing a sequence of values into components of different frequency, but computing it directly from the definition is very slow process. An FFT is a way to compute the same result more quickly: computing the DFT of N points in the naive way, using the definition, takes $O(N^2)$ arithmetical operations, while a FFT can compute the same DFT in only $O(N \log_2 N)$ operations.

1.2 DFTs In 3GPP LTE

In 3GPP LTE [15], data samples of different sizes are transmitted by multiple users at the same time. For example, as shown in the Fig 1, user 1 is transmitting N_1 data samples and user 2 is

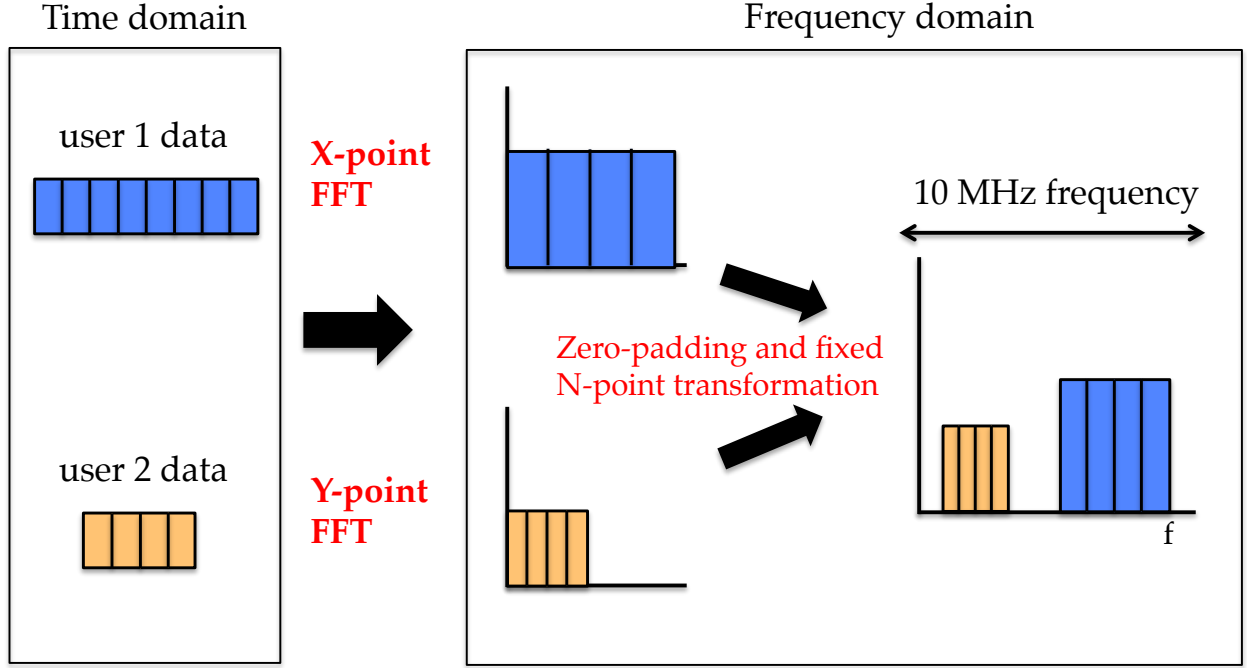


Figure 1: Multiband of SC-FDMA transmission In LTE

transmitting N_2 data samples. Transforming time domain samples into the frequency domain will result in the frequency domain plot as shown in the fig 1. Since the data samples transmitted by different users are of variable sizes, in order to transform the time domain samples to frequency domain, we need a mixed radix FFT scheme that can efficiently perform FFT operation.

If certain bandwidth has been allocated in the spectrum, shifting the signal in frequency domain and padding with zeros can eliminate interference between the users shown in Fig 1. Users are allocated different frequencies so that interference between various frequencies are avoided. In this way, we can allocate fixed bandwidth to N-number of users, utilizing effectively all the given bandwidth. There is often shifting involved in the frequency domain for any user X in order to send his signal without any interference. In LTE, we have different Block sizes transmitted by multiple users that can effectively utilize the entire allocated bandwidth by shifting and padding with zeros. So, there is a necessity to come up with a hardware that can handle different size FFTs or mixed radix FFTs.

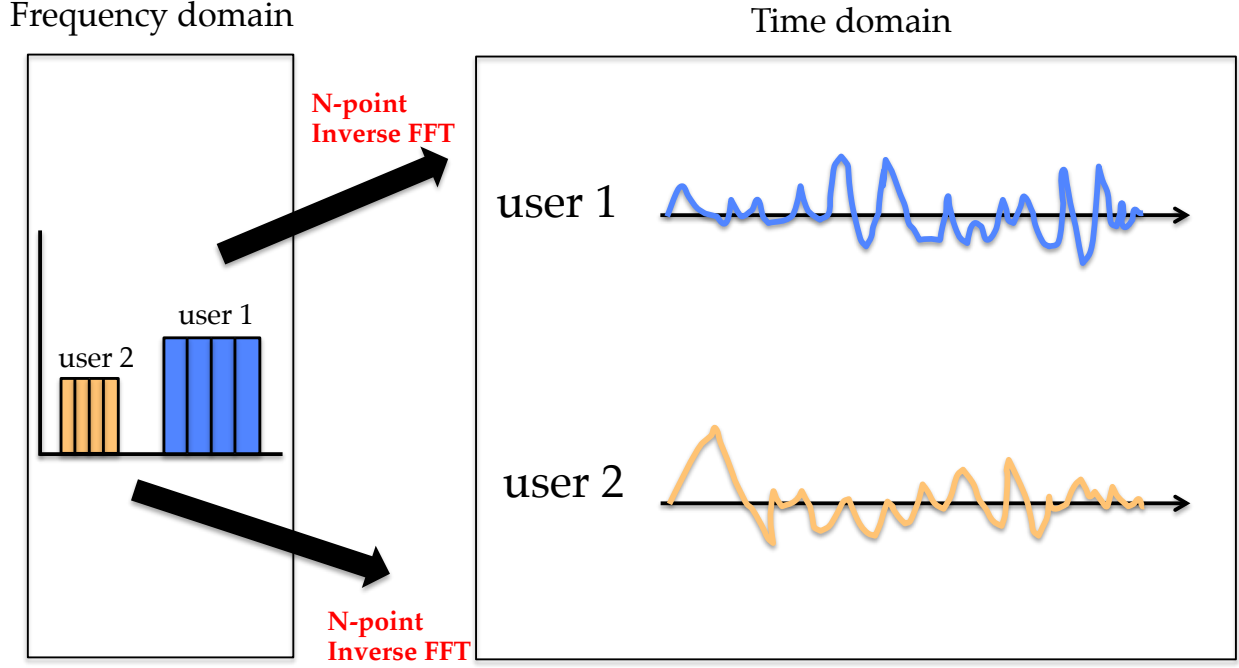


Figure 2: Frequency to time domain transformation

After transforming all the time domain samples into the frequency domain and allocating different frequency bands to different users which involves shifting in frequency domain, we apply a fixed N-point (already allocated frequency bandwidth for N users $N = N_1 + N_2 + N_3 + N_4 + \dots + N_n$) inverse-FFT to convert back the signal in time-domain. After converting the signal into the time domain, it is transmitted to the base station as shown in the Fig 2.

1.2.1 Receiver station for LTE

At the receiver side, we convert back the signal from time-domain to the frequency domain using a N-point fixed radix FFT [15]. All the individual components of frequencies by different users are retrieved in the Frequency domain shown in the fig 3. Each user converts back frequency domain signal to time domain using mixed radix inverse FFT for corresponding block sizes. This way, he will be able to retrieve back his original signal without any distortions.

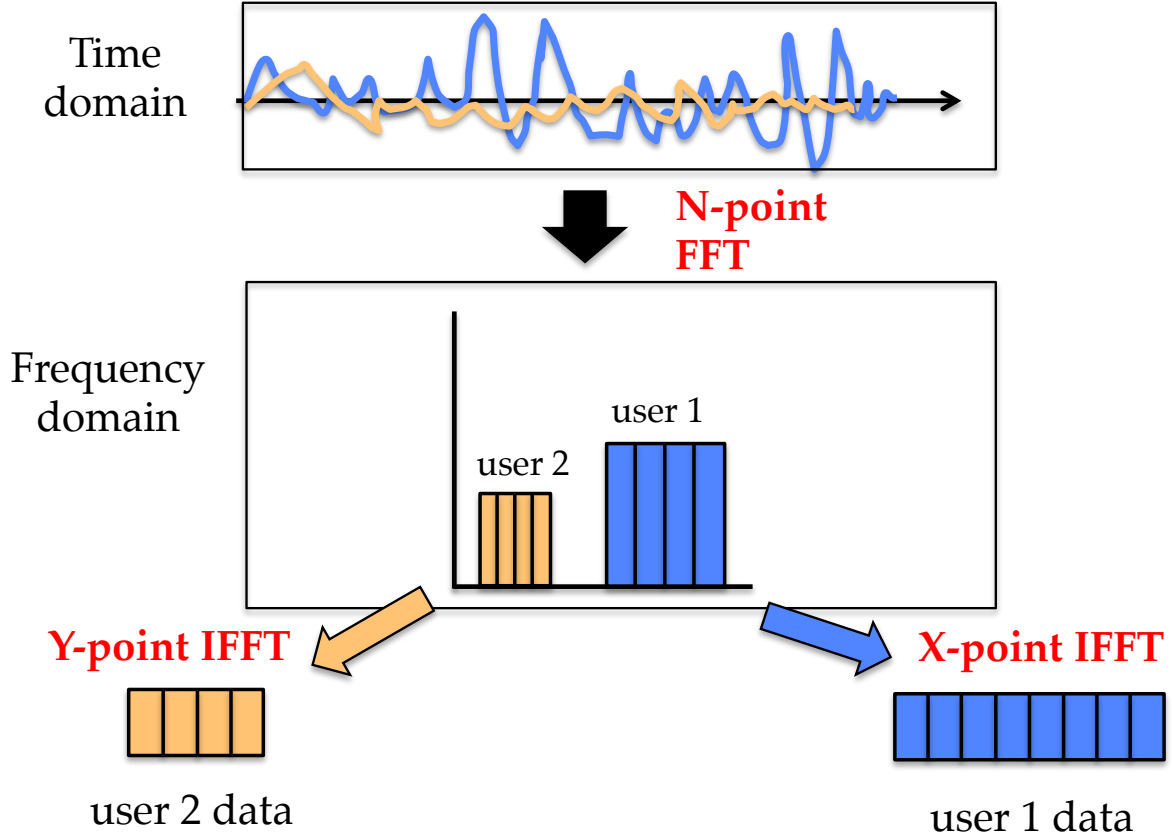


Figure 3: Time to frequency domain transformation

In the entire process, mixed radix FFT and IFFT play an important role at the transmitting and the receiving end. So, there is a necessity to come-up with an architecture for mixed radix FFT and IFFT that can efficiently perform these operations with improved speeds.

Block lengths [18] in the LTE standard are in the form of $2^m \times 3^p \times 5^q$. In total, there are 35 block lengths and there is necessity of coming up with fast, memory efficient and low area hardware. For reference, all the Block lengths are given as examples in fig 5

1 Introduction

N (Block length)	M (Radix-2)	P (Radix-3)	Q (Radix-5)
12	2	1	
24	3	1	
36	2	2	
48	4	1	
60	2	1	1
72	3	2	
96	5	1	
108	2	3	
120	3	1	1
144	4	2	
192	6	1	
216	3	3	
240	4	1	1
288	5	2	
300	2	1	2
324	2	4	
360	3	2	1
384	7	1	

Figure 4: Block lengths in LTE

1 Introduction

N (Block length)	M (Radix-2)	P (Radix-3)	Q (Radix-5)
432	4	3	
480	5	1	1
540	2	3	1
576	6	2	
600	3	1	2
648	3	4	
720	4	2	1
768	8	1	
864	5	3	
900	2	2	2
960	6	1	1
972	2	5	
1080	3	3	1
1152	7	2	
1200	4	1	2
1296	4	4	0
1536	9	1	0

Figure 5: Block lengths in LTE (Cont.)

2 Preliminaries

In literature, a variety of FFT algorithms exist of which the most basic algorithms are the decimation in frequency (DIF) and decimation in time (DIT). These basic algorithms are applicable for blocks for size 2^n , for any generic block lengths we mixed radix FFT scheme.

2.1 Decimation-in-time fast fourier transform

Fast Fourier transform is an algorithm that computes DFT with fewer computations, reducing the computation time. [1]FFT produces the exact same result as that of DFT, but is much faster than DFT. We are always referring to the complex domain whenever we talk about FFT of data points, all the sampled points are on the complex plane.

2.1.1 Frequency domain transformation equation

Consider $t[0], \dots, t[N-1]$ to be N complex numbers. The discrete fourier transform of this sequence of numbers is given by

$$F[k] = \sum_{n=0}^{N-1} t[n]e^{-i2\pi k \frac{n}{N}}, \text{ where } k = 0, \dots, N-1 \quad (2.1)$$

2.1.2 Mathematical complexity and periodicity

The expression defined above can be computed using $O(N^2)$ operations. There are N -outputs $F(k)$ after the transformation (refer 2.1), and it can be seen that every output is obtained through a summation of N -terms. In order to reduce the $O(N^2)$ computations, one can use the FFTs and they are capable of reducing the computations to $O(N \log_2 N)$. The best way to understand FFTs is to go through as many examples as possible.

Consider a signal with finite duration $t[n] \ n = 0, \dots, N-1$ its FFT was defined earlier 2.1.

Above expression 2.1 can be further simplified as

$$F[k] = \sum_{n=0}^{N-1} t[n] W^{nk} \quad (2.2)$$

where $W = e^{-j\frac{2\pi}{N}}$. It is easily seen that W^{nk} is periodic with a periodicity of N ; i.e.,

$$W^{(n+mN)(k+lN)} = W^{nk+nlN+mNk+mNlN} = W^{nk}, \text{ where } m, l = 0, \pm 1, \pm 2, \dots \quad (2.3)$$

The quantity W_N , corresponds to the twiddle factor and the periodicity of W^{nk} is an important factor to establish various forms of FFT.

From (2.1), it is clear that for a complex sequence to compute DFT for an N -point sequence from basic expression we need at least N^2 multiplications. As the value of N increases, the number of multiplications increase by a huge amount, we can't afford going for a naive implementation. The inordinate amount of computations involved forces us to come up with novel implementations that reduce the total number of operations. The FFT reduces an N -point sequence into two shorter sequences of even and odd samples. The odd and even samples can be combined to generate the original sequence.

2.1.3 Mathematics behind DIT transformation

A mathematical proof of reduced complexity by using an FFT is shown below. [1] Consider an N-point sequence $t(n)$ where $N = 2^x$, $x \in \mathbb{Z}$. We can also define $t_1(n)$ and $t_2(n)$ as even and odd sequence with $\frac{N}{2}$ -point each.

$$\begin{aligned} t_1[n] &= t[2n] \quad n = 0, 1, \dots, \frac{N}{2} - 1, \\ t_2[n] &= t[2n + 1] \quad n = 0, 1, \dots, \frac{N}{2} - 1, \end{aligned} \quad (2.4)$$

The DFT of an N-point signal $t[n]$ can be written as follows:

$$F[k] = \sum_{n=0}^{\frac{N}{2}-1} t[2n] W_N^{2nk} + \sum_{n=0}^{\frac{N}{2}-1} t[2n+1] W_N^{(2n+1)k} \quad (2.5)$$

Further simplifying the twiddle factor W_N^2

$$W_N^2 = (e^{-j(\frac{2\pi}{N})})^2 = e^{-j(\frac{2\pi}{N/2})} = W_{\frac{N}{2}} \quad (2.6)$$

Eq 2.5 can be written in the following form:

$$F[k] = \sum_{n=0}^{\frac{N}{2}-1} t_1[n] W_{\frac{N}{2}}^{nk} + W_N^k \sum_{n=0}^{\frac{N}{2}-1} t_2[n] W_{\frac{N}{2}}^{nk} \quad (2.7)$$

As a consequence, we have

$$F[k] = F_1[k] + W_N^k F_2[k] \quad (2.8)$$

2 Preliminaries

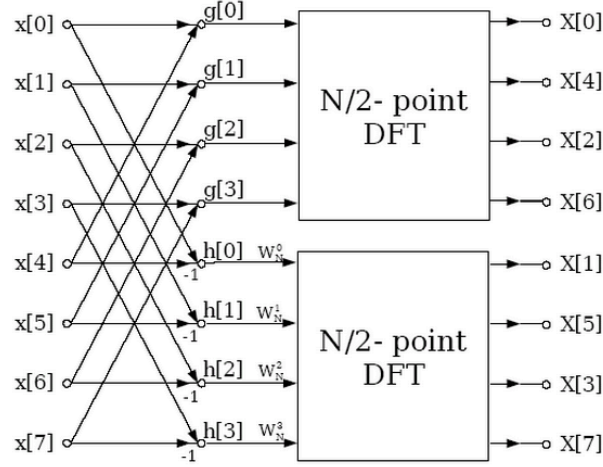


Figure 6: Division in 8-point FFT

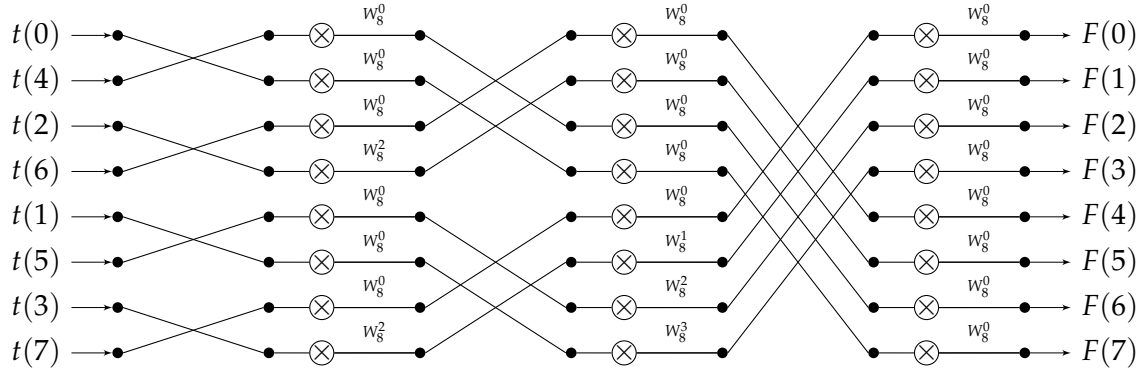
where $F_1(k)$ and $F_2(k)$ in the above equation are $\frac{N}{2}$ -point DFT's of even and odd sequence that have been defined earlier. A sequence of length N , where N is an even, the original sequence after breaking into two $(\frac{N}{2})$ -point sequences, would require $[\frac{N^2}{2} \cdot 2 = \frac{N^2}{2}]$ complex multiplications to compute the N -point DFT. This clearly shows a cost saving factor of 2 when compared to the naive approach. $\frac{N^2}{2}$ is the number of multiplications that are required for computing $\frac{N}{2}$ -point DFT and the multiplication factor 2 shows two such sequences. In a similar fashion Eq. 2.8, we can further make a recursive implementation and reduce $\frac{N}{2}$ -point DFT to two $\frac{N}{4}$ point DFT's (assuming $\frac{N}{2}$ is even) which will further reduce the computations by 50%. A reduction of 50% is only an approximate amount, since we are not taking into account computations involving combining all the smaller FFTs together.

It is clear from the Eq (2.8) that decomposing an N -point DFT into two $\frac{N}{2}$ -point DFT can later be combined according to the equation (2.8). The total number of complex multiplications that are required would be $(\frac{N}{2} + N)$ and as the value of N increases, savings would be close to 50% and it reduces the overall computation time. The frequency domain index is $X(k)$ valid for $0 \leq k \leq N - 1$ and the even and odd indices $F_1(k)$; $F_2(k)$ are defined for $0 \leq k \leq \frac{N}{2} - 1$.

Eight points are divided into two 4-point transforms (shown in fig.6) and each 4-point transform is further divided into 2-point transforms. A graphical representation of an 8-point DIT is shown in the Figure 7. [3] Since we are dividing the sequence into even and odd at every stage from the

2 Preliminaries

Figure 7: 8-point DIT example



output side, the inputs are expected to be in a different order. Hence, the concept of shuffling at the input side for normal output order. Consider $t_1(n)$ and $t_2(n)$ as even and odd sequence and their frequency domain transformations as $F_1(k)$ and $F_2(k)$ respectively. The sequence is divided into even and odd samples, and each set of samples are further divided into even and odd. $F_1(k)$, $0 \leq k \leq \frac{N}{2} - 1$ can be written as shown in 6 and the $\frac{N}{2}$ -point DFT can be expressed as a combination of $\frac{N}{2}$ -point DFT's, where we are recursively breaking down the sequence into even and odd.

$$F_1[k] = A[k] + W_{\frac{N}{2}}^k B[k] \quad (2.9)$$

$A[k]$ and $B[k]$ are $\frac{N}{4}$ -point even and odd DFTs of $t_1[n]$ that is defined in Eq 2.4 The graph in Fig 7 describes the division of four point DFTs that have been evaluated using the equation(2.9). The entire flow reduces an L-point DFT($L = 2^x$) to $\frac{L}{2}$ -point DFT, which can be recursively continued until we are left with two points.

2.2 Decimation-in-frequency algorithm

2.2.1 Mathematics behind DIF transformation

There is another well-established FFT known as the decimation-in-frequency (DIF). The approach differs from DIT as follows: Partitioning logic is different from when compared to the Decimation in Time. [1] The entire sequence $t[n]$ is divided into equal halves. The first half sequence is denoted by $t_1[n]$ and has $(\frac{N}{2})$ points of $t[n]$. On the other hand, the second half denoted by $t_2[n]$ covers the second half of the sequence $t[n]$, total number of points being $\frac{N}{2}$. Therefore,

$$\begin{aligned} t_1[n] &= t[2n] \quad n = 0, 1, \dots, \frac{N}{2} - 1, \\ t_2[n] &= t[n + \frac{N}{2}] \quad n = 0, 1, \dots, \frac{N}{2} - 1, \end{aligned} \tag{2.10}$$

Using the definition above, the N-point DFT of $t(n)$ is given by

$$\begin{aligned} F[k] &= \sum_{n=0}^{\frac{N}{2}-1} t[n] W_N^{nk} + \sum_{n=\frac{N}{2}}^{N-1} t[n] W_N^{nk} \\ &= \sum_{n=0}^{\frac{N}{2}-1} t_1[n] W_N^{nk} + \sum_{n=0}^{\frac{N}{2}-1} t_2[n] W_N^{(n+\frac{N}{2})k} \end{aligned} \tag{2.11}$$

Further simplifying the expression using the identity $W_N^{\frac{N}{2}} = e^{-j\pi}$

$$F[k] = \sum_{n=0}^{\frac{N}{2}-1} (t_1[n] + e^{-j\pi k} t_2[n]) W_N^{nk} \tag{2.12}$$

which follows from the identity $W_N^{\frac{Nk}{2}} = e^{-2\pi i k}$. $F[2k]$ and $F[2k+1]$ are the even and odd samples in the frequency domain that are defined as follows:

2 Preliminaries

$$\begin{aligned}
 F[2k] &= \sum_{n=0}^{\frac{N}{2}-1} (t_1[n] + t_2[n]) (W_N^2)^{nk} \\
 &= \sum_{n=0}^{\frac{N}{2}-1} [t_1[n] + t_2[n]] (W_{\frac{N}{2}}^{nk})
 \end{aligned} \tag{2.13}$$

$$\begin{aligned}
 F[2k+1] &= \sum_{n=0}^{\frac{N}{2}-1} [t_1[n] - t_2[n]] W_N^{n(2k+1)} \\
 &= \sum_{n=0}^{\frac{N}{2}-1} ([t_1[n] - t_2[n]] W_N^n) W_{\frac{N}{2}}^{nk}
 \end{aligned} \tag{2.14}$$

Eq 2.13 and 2.14 show that the even and odd valued samples of the DFT can be obtained from the $\frac{N}{2}$ -point DFT's of the sequences $f(n)$ and $g(n)$ respectively [1], where

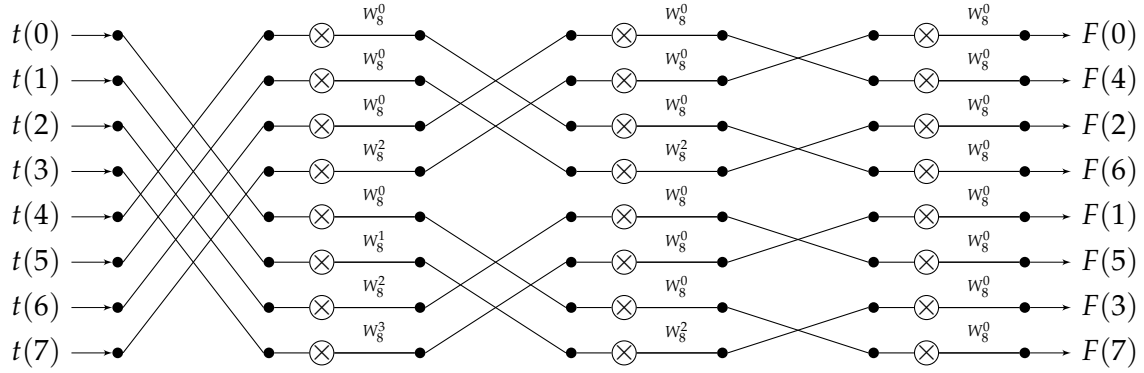
$$\begin{aligned}
 f(n) &= T_1(n) + T_2(n) \quad n = 0, 1, \dots, \frac{N}{2} - 1, \\
 g(n) &= [T_1(n) - T_2(n)] W_N^n \quad n = 0, 1, \dots, \frac{N}{2} - 1,
 \end{aligned} \tag{2.15}$$

Reducing the problem from an N -point DFT to two equal $\frac{N}{2}$ -point DFT. Shown in the figure (ref) is the procedure for $N=8$. In this way, [3] we can repeat the entire procedure for each $\frac{N}{2}$ -point DFT, writing them as a combination of two $\frac{N}{4}$ -point DFT. As shown in the Figure (ref), we are expressing the four-point DFT as two 2-point DFTs.

2.2.2 Difference between DIT and DIF

We can see two differences after comparing computational algorithms in Fig 7 with Fig 8. [1] The Decimation in Time which we discussed earlier has shuffled inputs, and the outputs are in normal order. On the other hand, Decimation in Frequency has inputs in normal order and the outputs are in shuffled manner as shown in Fig 8. The notable difference is that the multiplication takes

Figure 8: 8-point Decimation in Frequency



place after the add-subtract operation in DIF. There are huge similarities between DIF and DIT, both the algorithms require computations in the order of $(N \log N)$ to compute DFT. Bit-reversal is either at the input end or the output end for both these algorithms.

2.3 Generalized approach to FFT in mixed radix form

So far, we considered DFT sizes of length $N = 2^x \times \in \mathbb{Z}$. In general, for applications in LTE, we deal with the DFT samples of sizes that are not powers of 2, in these cases mixed radix approach is the best way to calculate FFT. The main idea behind all mixed radix schemes is to transform the data points from one dimensions to multi-dimensions, which means we can represent one dimensional data samples as multiple dimensions data samples. In this section, we are discussing the Unified approach that can be applied to all radices at the same time. All the numbers that have been discussed so far were composite, but prime numbers cannot be expressed as the product of smaller integers. [1] A one dimensional signal is $x(0), x(1), \dots, x(N-1)$ can be can be decomposed into multiple dimensions until we reach a point where we can't further decompose. A prime number cannot be decamped further into smaller integers.

2.3.1 Basis behind mapping from one dimension to two dimensions

We express the DFT samples $X(k) = X(s, r)$ as the transform of $x(n) = x(l, m)$ (s, r, l, m have already been defined) by simply substituting (2.20) and (2.21) into the definition of DFT, which gives.

$$X[k] = X[s, r] = \sum_{m=0}^{M-1} \sum_{l=0}^{L-1} x[l, m] W^{[Ml+m][Lr+s]} \quad (2.16)$$

Using the simplification $W^{MLr} = W^{Nlr} = 1$ on $W^{(Ml+m)(Lr+s)}$, we further simplify (2.16)

$$X[k] = X[s, r] = \sum_{m=0}^{M-1} W^{Lmr} W^{ms} \sum_{l=0}^{L-1} x[l, m] W^{Msl} \quad (2.17)$$

Our final 2.17, is the key to establish a formal procedure for evaluating one-dimensional transforms through two dimensional representations. [1] First, we are computing the L-point DFT of each column and then multiplication by a twiddle factor. The result after the first stage is a function denoted as $q(s, m) = \sum_{l=0}^{L-1} x[l, m] W^{Msl}$, where m varies from 0 to $M-1$. Simplified form of the (2.17) is as follows.

$$X[s, r] = \sum_{m=0}^{M-1} W^{Lmr} W^{ms} q[s, m] \quad (2.18)$$

From (2.18), next step in the procedure is deduced to be. We obtain a new array $h(s, m) = W^{ms} \sum_{l=0}^{L-1} x[l, m] W^{Msl}$ by multiplying every $q(s, m)$ with a twiddle factor exponent.

Equation (2.18) now reduces to

$$X[s, r] = \sum_{m=0}^{M-1} h[s, m] W^{Lmr} \quad (2.19)$$

2.3.2 Examples with detailed analysis

With a basic example, we can understand the principles of a mixed radix transform. Consider $N = 12$, we can express 12 as a product of smaller integers in many ways, e.g. $12 = 3 \times 2 \times 2 = 2 \times 2 \times 3 = 2 \times 3 \times 2$. The order of decomposition will be important and different orders result in different flow diagram. Mixed radix can be defined as the factors after decomposition that are not identical. Consider a number that can be divide into only two prime numbers, in any case it forms a mixture of two different numbers when compared to N -point transform which can be decomposed into series of same numbers. Any decomposition algorithm is called a radix- r algorithm only when all forms of decomposition are results in the repetition of the same integer r . For example, if $N = 32 = 2 \times 2 \times 2 \times 2 \times 2$, we have the radix-2 algorithms that we discussed in the previous sections. While decomposing N -data points, it is important to perform a stage wise decomposition and stage wise decomposition helps us in building matrix in which the rows can be further decomposed into smaller and smaller points. Consider $N = 12$ as an example, it can be expressed as 3×4 and then express 4 as 2×2 . In the same way, it could be expressed as $12 = 2 \times 6$ and $6 = 3 \times 2$. Successive decomposition is the key while working with the mixed radix schemes.

0	1	2	3	4	5
6	7	8	9	10	11

In the above table, we have expressed signal indices as a (2×6) matrix. Now since, each column has 2 samples (a prime number), they can be decomposed no further; however, each 6-sample row can be expressed as a (2×3) matrix. Consider the first row as an example.

0	1	2
3	4	5

All the other rows can be expressed similarly. Thus, we are simplifying a huge problem N - point DFT into the smaller DFT's just by rearranging them in different form. To generalise the which has been shown above, we need to represent entries in the matrix as a function of row index and a column index. This can easily be done by replacing each entry by a pair of entries as follows:

2 Preliminaries

$$\begin{array}{cccccc} 0,0 & 0,1 & 0,2 & 0,3 & 0,4 & 0,5 \\ 1,0 & 1,1 & 1,2 & 1,3 & 1,4 & 1,5 \end{array}$$

Let the column index be $m(= 0,1)$ and the row index be $l (l = 0,1,...,5)$ and if n is the original index ($n = 0,1,2...12$), then we see that

$$n = Ml + m \text{ where } M = 6 \quad (2.20)$$

The number of columns in the example above is $M = 6$. After performing the DFT on the two dimensional array, the result would be again a two dimensional doubly-indexed array. Let r and s be the transformed column and row variables, m and l are variables in frequency domain. Decomposing frequency domain in two variables as shown below.

$$k = Lr + s \text{ where } L = 2 \quad (2.21)$$

2.3.3 Row and Column Permutations

We notice that (2.19) is the M -point DFT of each row. Next and final step is to compute the Discrete Fourier Transform of each row in $h(s,m)$ function which was described before. The twiddle factor in this process is W^L . The procedure discussed so far is related to calculating the two dimensional DFT. We compute row DFTs followed by the column DFTs to find the answer.

We always compute DFT of lower dimension first, then go for a higher dimension as it is computationally less intensive, [1] because larger dimensions can be further decomposed into further smaller dimensions. While transforming a single indexed array into a two dimensional array, we can simply distribute elements across the matrix, but this operation introduces twiddle factors in between stages. The twiddle factor multiplication step introduces at the most N -multiplications for a transformation from one-dimension to two dimensional array. The order of twiddle factor

2 Preliminaries

$$\begin{array}{llll} X(0) = X(0,0) & X(3) = X(0,1) & X(6) = X(0,2) & X(9) = X(0,3) \\ X(1) = X(1,0) & X(4) = X(1,1) & X(7) = X(1,2) & X(10) = X(1,3) \\ X(2) = X(2,0) & X(5) = X(2,1) & X(8) = X(2,2) & X(11) = X(2,3) \end{array}$$

Table 1: frequency domain arrangement

multiplication depends on the whether we are dealing with DIT or DIF algorithms for radix-2 FFTs. From (2.19), it is evident that variables m and r are column indices, l and s are row indices. If we increment variable m by 1, the signal index $(Ml + m)$ is incremented by 1. On the other hand, incrementing the transformed variable l column index by 1, increments the argument of $x(s,r)$ by L . Using this result, we can see that the row and column index undergo a permutation as a result of this transformation.

Consider 12-point example for permutation analysis, the decomposition order would be 4×3 . Time domain matrix can be written as

$$\begin{array}{lll} x(0) = x(0,0) & x(1) = x(0,1) & x(2) = x(0,2) \\ x(3) = x(0,3) & x(4) = x(0,4) & x(5) = x(1,0) \\ x(6) = x(1,1) & x(7) = x(1,2) & x(8) = x(1,3) \\ x(9) = x(1,4) & x(10) = x(2,0) & x(11) = x(2,1) \end{array}$$

On the frequency domain, the matrix would be reflected as follows.

Consider $N = 30 = 5 \times 6 = 5 \times 2 \times 3$, there are two stage decompositions involved in calculating the DFT. We start with 5×6 and further decompose the 6-point into 2×3 , in this way, we will be able to calculate the entire 30 point DFT [3].

First level decomposition

2 Preliminaries

$$\begin{array}{llllll}
0 = (0,0) & 1 = (0,1) & 2 = (0,2) & 3 = (0,3) & 4 = (0,4) & 5 = (0,5) \\
6 = (1,0) & 7 = (1,1) & 8 = (1,2) & 9 = (1,3) & 10 = (1,4) & 11 = (1,5) \\
12 = (2,0) & 13 = (2,1) & 14 = (2,2) & 15 = (2,3) & 16 = (2,4) & 17 = (2,5) \\
18 = (3,0) & 19 = (3,1) & 20 = (3,2) & 21 = (3,3) & 22 = (3,4) & 23 = (3,5) \\
24 = (4,0) & 25 = (4,1) & 26 = (4,2) & 27 = (4,3) & 28 = (4,4) & 29 = (4,5)
\end{array}$$

We decompose each row into smaller point DFTs and then multiply with twiddle factors. The resulting matrix $p(s,m)$ is shown below.

First level decomposition - without indices

$$\begin{array}{llllll}
p(0,0) & p(0,1) & p(0,2) & p(0,3) & p(0,4) & p(0,5) \\
p(1,0) & p(1,1) & p(1,2) & p(1,3) & p(1,4) & p(1,5) \\
p(2,0) & p(2,1) & p(2,2) & p(2,3) & p(2,4) & p(2,5) \\
p(3,0) & p(3,1) & p(3,2) & p(3,3) & p(3,4) & p(3,5) \\
p(4,0) & p(4,1) & p(4,2) & p(4,3) & p(4,4) & p(4,5)
\end{array}$$

Shown below is how each six-point row is decomposed into a (2×3) matrix. Thus, for example, the first two rows of the h-matrix becomes.

Row 1

$$\begin{array}{lll}
p(0,0) & p(0,1) & p(0,2) \\
p(0,3) & p(0,4) & p(0,5)
\end{array}$$

Row 2

$$\begin{array}{lll}
p(1,0) & p(1,1) & p(1,2) \\
p(1,3) & p(1,4) & p(1,5)
\end{array}$$

[3]6-point transform in each row can now be further decomposed into 2×3 . Calculating a 2-point Transform of each column followed by multiplication with a twiddle factor, and then later transforming three-point rows is simpler. When this is completed for all the five of the rows, we are

done computing 30-point FFT.

2.4 30 point FFT using mixed radix

2.4.1 Row and column divisions

Here we have decomposed a 30-point array into a 5-row 6-column two dimensional array, as shown below (numbers are representing the indexes of the input sequence).

The first FFT stage calculates six five-point DFTs. Each radix unit has inputs going into it, and outputs coming out of it. The number of inputs entering a radix unit is same as the number of output coming out of the radix unit. The nodes in this flow graphs are the registers that hold the input and outputs of the DFTs. [1] After computing 5-point DFTs, all the outputs are twiddled as shown in the figure 9 by 30 arrows with W 's (indicating twiddle factors). The next step in the procedure is to compute row transforms. Since each row contains six elements, each row can be further decimated into a 2-rows and three columns matrix. Again, we are following same method for 6-point transform, we are breaking it down into three radix-2, twiddling it, later simply calculating radix-3 in order to get the result. Every line that is coming out of the radix unit has a twiddle factor, if the twiddle factor happens to be one, it is left out. Figure below shows the first level decomposition.

0	1	2	3	4	5
6	7	8	9	10	11
12	13	14	15	16	17
18	19	20	21	22	23
24	25	26	27	28	29

2.4.2 Algorithm in detail for a 30-point FFT

Let us discuss the algorithm in detail, we can take an example shown above to illustrate this algorithm. We are fixing the order in which we decompose our FFT. We are considering a 30-point FFT. Let us follow the decomposition order as $5 \times 2 \times 3$ and process elements accordingly. In the first stage, we are using radix 5 to compute FFT. In this stage, we decompose 30 into 5×6 , six radix-5 units are required to calculate 30 point FFT in the first stage. As shown in the above matrix, indices that are required as inputs for the first radix-5 unit can be collected from the first column like 0 6 12 18 24, these five indices are processed by the radix unit simultaneously as inputs. Similarly, FFTs of all the other columns are computed.

Fig 10 shows clearly number of inputs for radix-5 unit and outputs. After processing each radix-5 unit, twiddle factor is multiplied with each of the outputs as shown in the Fig 10. Outer index exponent in Fig 10, increases with the n^{th} radix-5 unit operating on that.

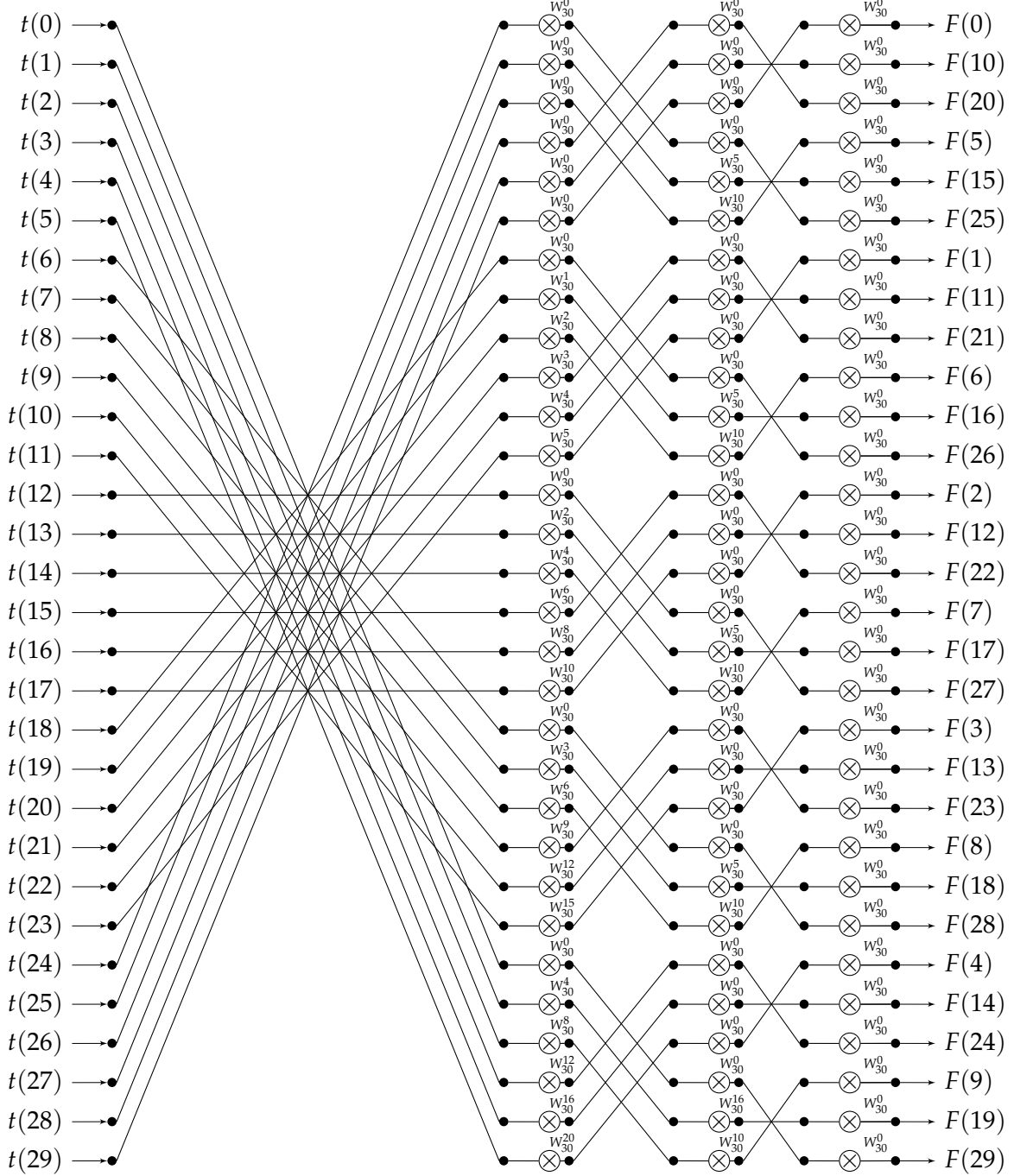
2.4.3 Radix calculations

As shown in the Fig 9 there are six radix-5 units, $R_{50}, R_{51}, R_{52}, R_{53}, R_{54}, R_{55}$. For the first radix R_{50} , as shown in the fig 10 each of the output is multiplied by $(W_{30}^0)^0 (W_{30}^1)^0 (W_{30}^2)^0 (W_{30}^3)^0 (W_{30}^4)^0$. Similarly, for the second radix R_{51} , output is multiplied by $(W_{30}^0)^1 (W_{30}^1)^1 (W_{30}^2)^1 (W_{30}^3)^1 (W_{30}^4)^1$. The variable index that changes from R_{50} to R_{51} is the outer index (outer exponent) of the twiddle factor that is shown clearly in the Fig 9. Same analogy can be extended for other radices, as you can see that the outer index changes from one radix to the other in the increasing order.

As soon as we compute with the first stage, each row in the matrix that has been described earlier should be our focus in the second step, since we have reduced one dimensional problem into two dimensions. Consider 0 1 2 3 4 5, we are using radix-2 unit in the second stage, hence dividing it into 2 rows now ends up with an arrangement shown below.

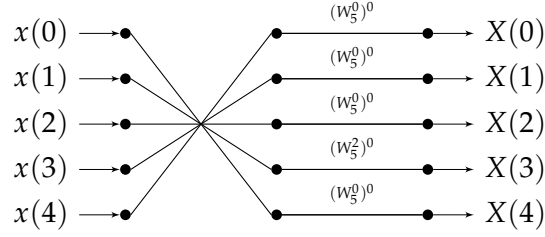
Division of each row in the second stage.

Figure 9: 30-point mixed radix example using row and column permutations



2 Preliminaries

Figure 10: Radix-5 example



0 1 2
3 4 5

2.4.4 Simplification to common index

Each column in the matrix above will be an input to the radix-2 unit and three such radix-2 units are used in the second stage at each tree level decomposition as shown in the Fig. 11. Let us represent the radix units by R_{20}, R_{21}, R_{22} . Each radix unit has two outputs and the twiddle factors for the first radix-2 unit would be $(W_6^0)^0 (W_6^1)^0$. Similarly, for the second radix unit, the twiddle is $(W_6^0)^1 (W_6^1)^1$. Since its 30 point implementation example, it is always to simple to use a common index for all the twiddle. Conversion from one index to the other is shown below. Hence, following modifications can be applied to the second stage.

$$W_N = (e^{-j(\frac{2\pi}{N})})$$

We can deduce following from the above established equivalence

$$(W_{30}^0)^0 = (W_6^0)^0$$

$$(W_{30}^5)^0 = (W_6^1)^0$$

$$(W_{30}^0)^1 = (W_6^0)^1$$

$$(W_{30}^5)^1 = (W_6^1)^1$$

2 Preliminaries

The last stage in our three stage approach would be to compute radix-3 unit on each of the second stage rows, which is a recursive idea. Each row of the matrix that is shown above forms an input to the radix-3 unit, hence inputs for the radix three unit have indices 0 1 2.

We are reducing the problem from a single dimension into multiple dimension and focusing on each dimension while they are separated. In our 30-point example, we first use radix-5 for all 30 points, then radix-2 for each 6 points, finally radix-3 for last three points. In this way, we are decomposing the problem into smaller problems and then solving it independently. Our approach is shown in Fig 11

Similarly, we can consider one more simple case and analyse the graph flow from one stage to the other . Since we are only [3] discussing a 6 point FFT, we can decompose it into 2×3 (different order than the earlier example). In the first stage, we are using radix-2, and form a matrix as shown in below (matrix of order 2×3)

Decomposition in the first Level:

$$\begin{array}{ccc} 0 & 1 & 2 \\ 3 & 4 & 5 \end{array}$$

Each column now forms the input to radix-2 unit. Input elements for the first radix-2 unit would be 0 3 indices of the input. And input to the second radix-2 unit would be indices 1 4. Each of the three outputs from the first radix unit are multiplied by $(W_6^0)^0, (W_6^1)^0$ and the second radix unit output is multiplied by $(W_6^0)^1, (W_6^1)^1$. Multiplication of the twiddle factors is shown in the Fig 12.

In the second stage of the algorithm, each and every row in the first stage of the algorithm is now independent and processed separately. For example, indices 0 1 2 ; 3 4 5 are independent of each other. Radix-3 unit operate on each of them and twiddle factors would be $(W_3^0)^0 (W_3^1)^0 (W_3^2)^0$. They are equivalent to 1, so need not be represented on the output. But, in order to unify everything to the N-point index (N = 6 in this case), we can equate following.

Figure 11: 30-point FFT showing decomposition

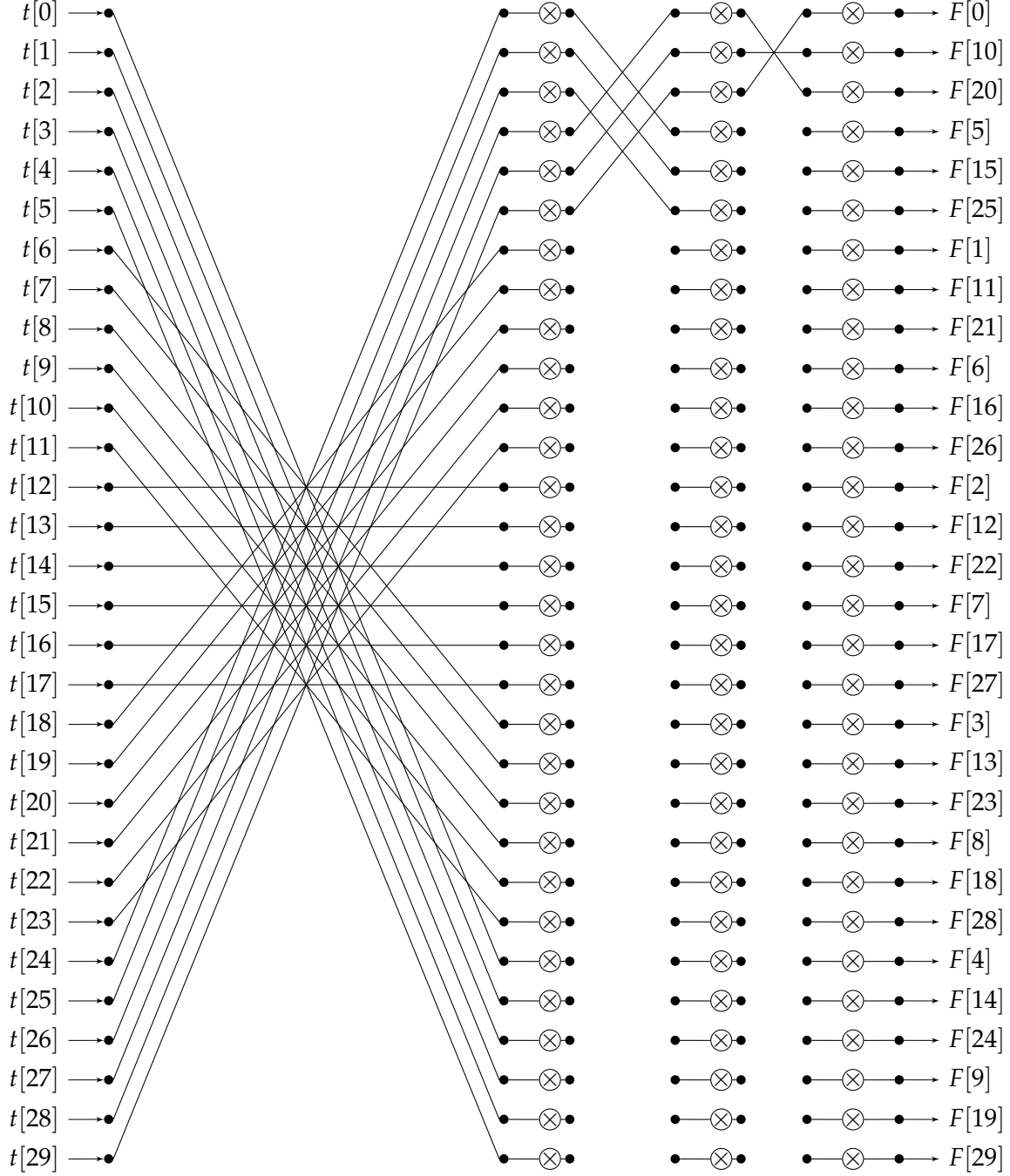


Figure 12: 6 point mixed radix FFT

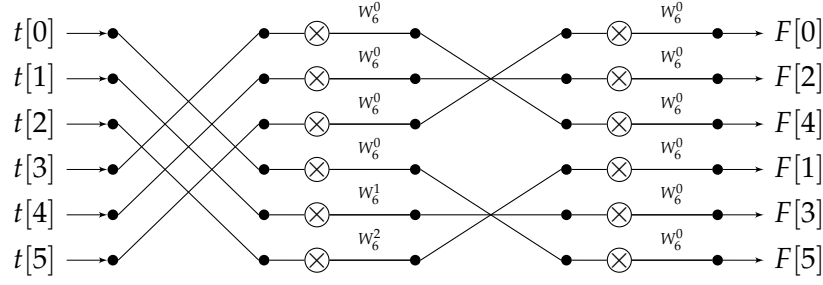
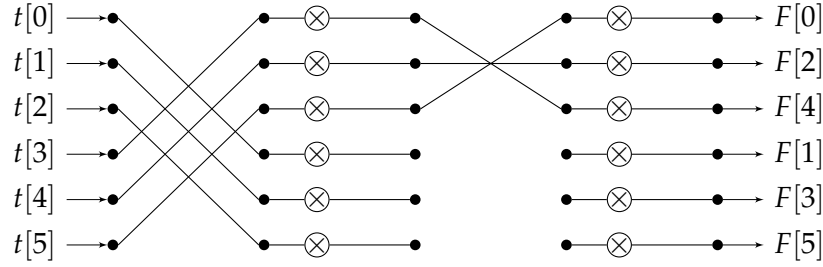


Figure 13: 6-point decomposition



Converting all twiddle factors to a single index:

$$(W_2^0)^0 = (W_6^0)^0$$

$$(W_2^1)^0 = (W_6^1)^0$$

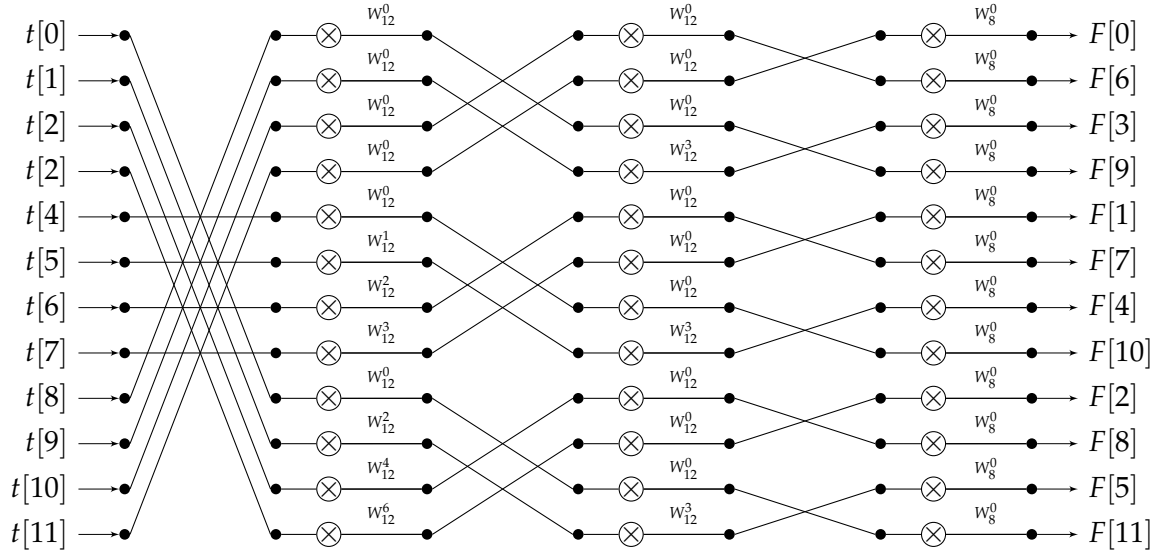
We are recursively decomposing the problem into smaller problems, 6 point FFT in the first stage and 3 point FFT in the second stage as shown in the Fig 13.

2.5 Data shuffling in mixed-radix implementations

2.5.1 General formula

One of the most important factor we are missing in our discussion is the digit-reversed data output. [19] If samples in the time domain are inputs in the FFT transformation, the order of outputs in the frequency domain depends on the order at which the FFT is performed. For example, let us consider a simple case of 6 point FFT. We use radix-2 and radix-3 units to process 6 point FFT. 6

Figure 14: 12-point Bit reversal example



point FFT can be computed in 2×3 order or can be computed in 3×2 . Each of the above order results in different shuffled versions of the frequency domain output.

Let us consider a simple example of 12-point FFT graph flow shown below (order $3 \times 2 \times 2$). The time domain input samples are in normal order (0,1,2,3,..11) and the output symbols are in different order as shown in Fig 14.

The inputs are in normal order and the outputs are digit-reversed order. Bit reversal can be done either ways, if we can store the inputs in digit-reversed order, output produced will be in normal order. Usually, the digit reversed order at the inputs is stored and pre-calculated outside the DFT function. The digit reversed order n can be calculated in the following way

Input [19] is the product of radices and the output is the order itself (Input and the outputs are clearly defined below the equation):

$$n = b(0)C(0) + b(1)C(1) + b(2)C(2) + b(3)C(4) + \dots + b(S-1)C(S-1) \quad (2.22)$$

where S is the total number of stages, $b(s) = \{0,1,2,..,B(S-1)\}$ specifies possible digits of Radix $B(s)$,

and $C(s) = \prod_{i=1}^k B_i$; k is the number of stages in the transform, specifies the product of radix of the following stages (This will be clear with an example).

2.5.2 Example for index shuffling at the output

For example, let us consider a 12-point FFT which can be computed using radix-2 and radix-3. 12 point DFT can be factored as $3 \times 2 \times 2$. In order to derive the bit reversed order in frequency domain, we usually reverse the order i.e, $2 \times 2 \times 3$, and consider it as our base reference. The product of radices of following FFT stages are also considered to compute the formula.

$C(i)$ calculation in the 12 point FFT is shown below. All variables in the formula for output index can be computed in the following way

First stage, $C(0) = 2 \times 3 = 6$, $b(0) = \{0,1\}$

Second stage, $C(1) = 3$, $b(1) = \{0,1\}$

Third stage, $C(2) = 1$, $b(2) = \{0,1,2\}$

The final output indices in the digit-reversed order are calculated as shown in the table 2.

Table 2 clearly shows output in frequency domain which is bit reversed where the input in time domain is in normal form.

On the other hand, if the input in time domain is in bit reversed form, the output in the frequency domain will be in normal form. To illustrate this concept, following table 3 helps. Consider a digit-reversed order of 300-point FFT. 300 point FFT can be factored in $4 \times 3 \times 5 \times 5$ and the calculated radix of FFT is performed in a similar way. Product of radices of following FFT stages are:

2 Preliminaries

Table 2: Digit-reverse in Frequency domain

b(0)C(0)	b(1)C(1)	b(2)C(2)	k
0 +	0 +	0	= 0
6 +	0 +	0	= 6
0 +	3 +	0	= 3
6 +	3 +	0	= 9
0 +	0 +	1	= 1
6 +	0 +	1	= 7
0 +	3 +	1	= 4
6 +	3 +	1	= 10
0 +	0 +	2	= 2
6 +	0 +	2	= 8
0 +	3 +	2	= 5
6 +	3 +	2	= 11

Variables in the formula are calculated in a similar way as shown below. Every stage $C(i)$ changes and its derivation is shown below:

First stage, $C(0) = 3 \times 5 \times 4 = 75$, $b(0) = \{0,1,2,3\}$

Second stage, $C(1) = 5 \times 5 = 25$, $b(1) = \{0,1,2\}$

Third stage, $C(2) = 5$, $b(2) = \{0,1,2,3,4\}$

Fourth stage, $C(3) = 1$, $b(3) = \{0,1,2,3,4\}$

2 Preliminaries

Table 3: Digit-reverse in Time domain

T	b(0)C(0)	b(1)C(1)	b(2)C(2)	b(3)C(3)
0 =	0 +	0 +	0 +	0
75 =	75 +	0 +	0 +	0
150 =	150 +	0 +	0 +	0
225 =	225 +	0 +	0 +	0
25 =	0 +	25 +	0 +	0
100 =	75 +	25 +	0 +	0
175 =	150 +	25 +	0 +	0
250 =	225 +	25 +	0 +	0
50 =	0 +	50 +	0 +	0
125 =	75 +	50 +	0 +	0
200 =	150 +	50 +	0 +	0
275 =	225 +	50 +	0 +	0
5 =	0 +	0 +	5 +	0
...				

3 Hardware architecture for single memory mixed radix FFT and idea of banking

The hardware model of mixed radix scheme that has been discussed so far has been is shown in Fig 15. [7] Single port memories have only one port for data read at a time, for radix- r unit, we need r data values in parallel to process elements simultaneously. If we read elements serially, there is always an extra latency for every input. Processing elements serially (Fig 15) reduces utilization and degrades performance of the system. There is a necessity to come up with a hardware that can read elements simultaneously and process them in the radix unit. Moreover, the idea of having separate radix units (2,3,5) takes extra space or hardware on the actual chip. [8] The idea of accessing all the elements in parallel will give a greater throughput and can potentially improve speed by a huge amount.

Whenever we read data simultaneously, [8] we need to ensure that r -inputs in a radix- r unit should be available in different banks. This is defined as Conflict Free reading scheme where all the inputs are present in as different banks locations. So, there is a necessity to come up with conflict free reading scheme, which means that all the elements from single port memory will be mapped into different banks such that the elements are processed at the same time. This is called *Conflict Free Mapping* and there are no memory contentions in this process. It is shown clearly in the fig 17 below. Conflict free processing should ensure that the current processing stage and the stages later are conflict free.

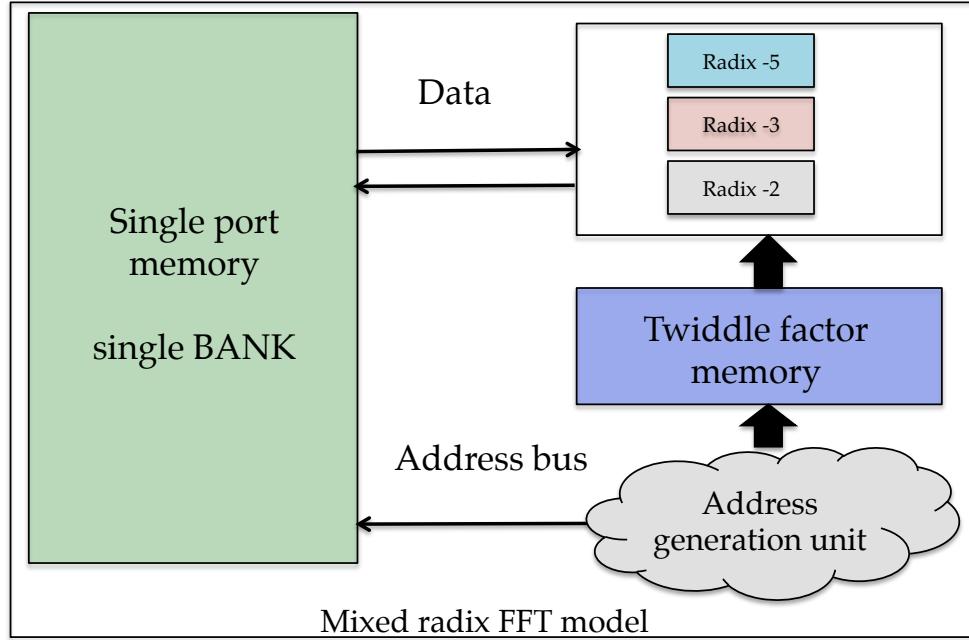


Figure 15: Hardware architecture for single memory mixed radix FFT

Consider a twelve-point example as shown in Fig 18. In the first stage, we have radix-3 units and each of them require three elements simultaneously to process, so we distribute them into three different banks as shown in the Fig 18 below with different colours. We can see in the second stage that the radix-2 unit again have memory conflicts. So, simple division of elements in the first stage into different banks does not work. We need to ensure there exists a mapping such that the current stage elements and the stages later, will not have any memory conflicts. So, a unique mapping is yet to be defined so it can avoid all the conflicts that can happen.

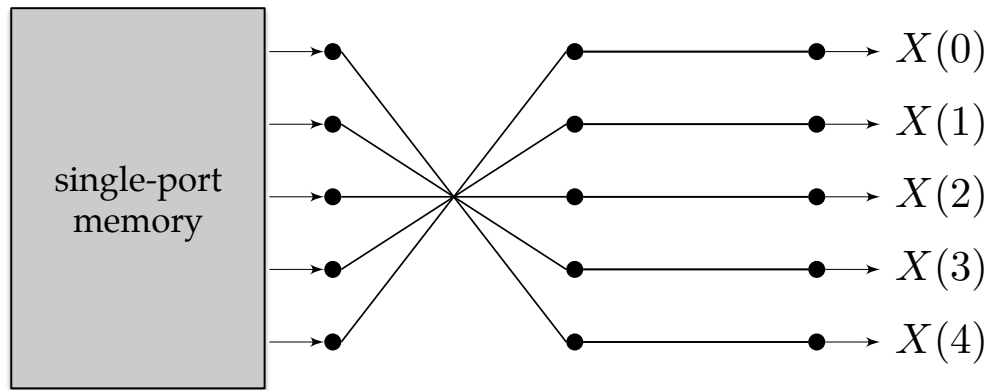


Figure 16: Reading issues associated with single port memories

3 Hardware architecture for single memory mixed radix FFT and idea of banking

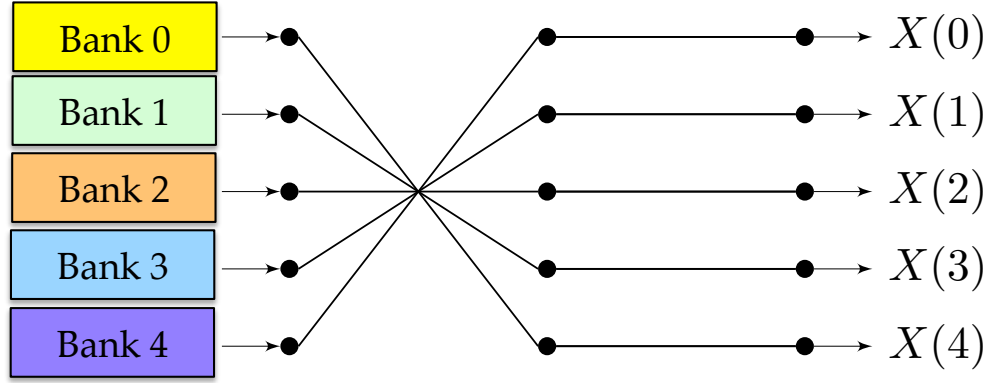


Figure 17: Conflict free mapping without memory conflicts

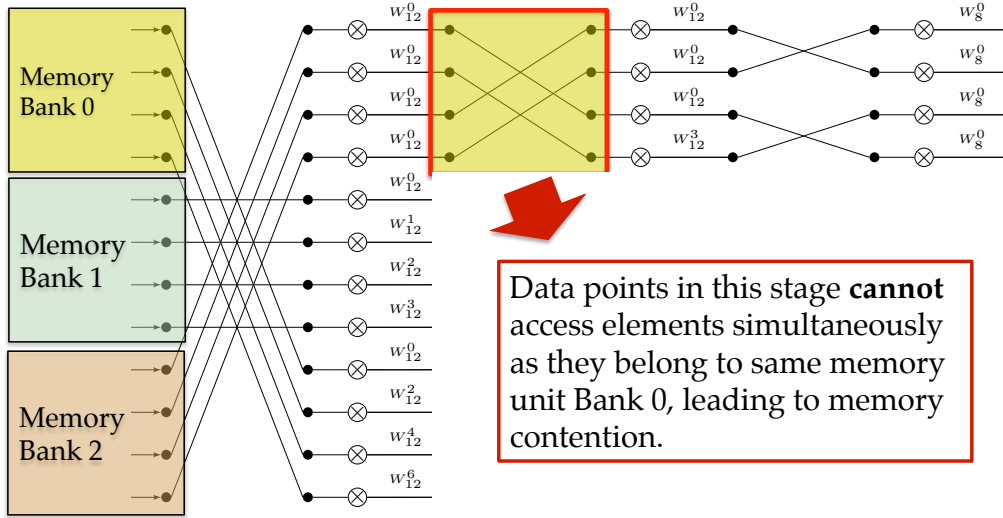


Figure 18: Problem persists even after bank division

We need to define a transformation shown in Fig 19 on the input points such that the current stage and the subsequent stages will not have any address conflicts after distributing data points into different banks. The next chapter discusses algorithmic implementations and its disadvantages, followed by a method to map elements into different banks and avoid memory contentions. Initially, we discuss mapping techniques for fixed radix algorithms and then a further complex mapping schemes for mixed radix algorithms.

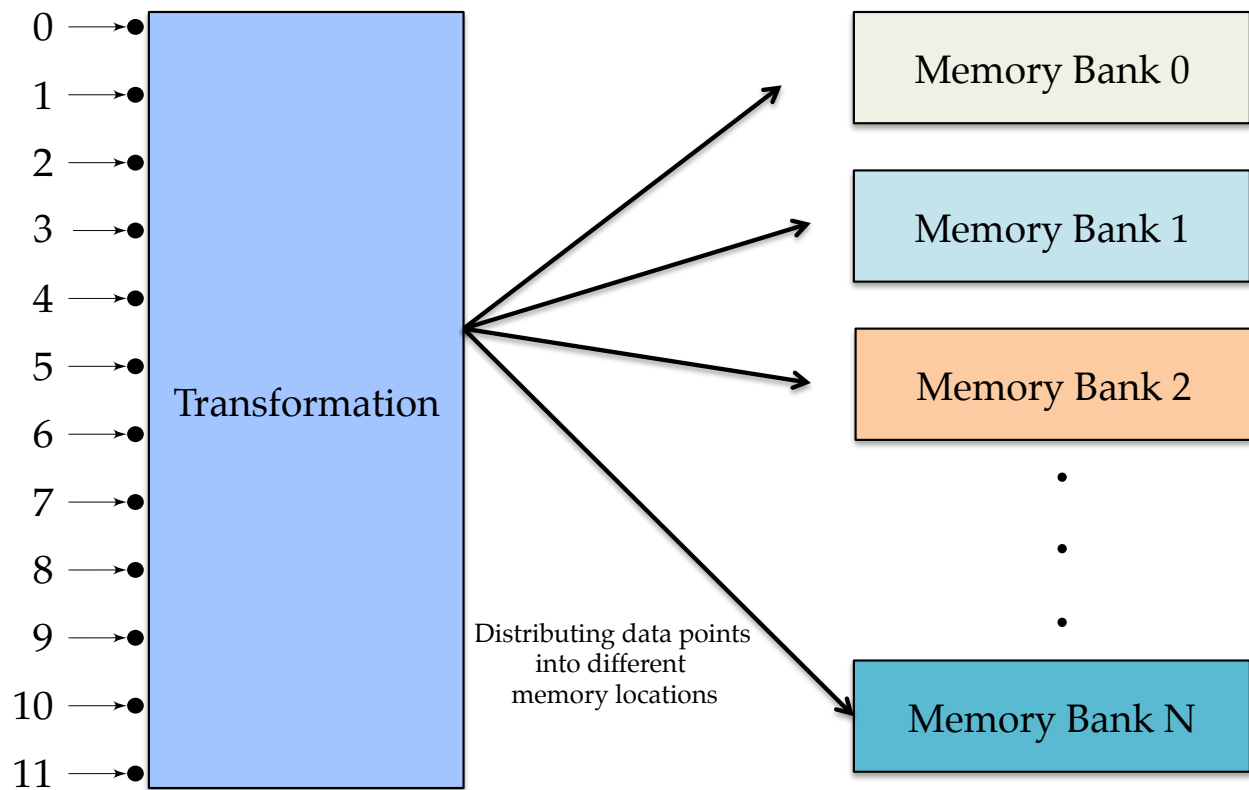


Figure 19: Ideal solution for multi bank Approach

4 Discussion / Algorithms

Fast Fourier Transform is computed by [1] reconstructing a Discrete Fourier Transform of large number N of sample data points using transforms of smaller data. Smaller FFT transform are termed as smaller radix units often helpful in finding a larger N -point transform. The mixed radix FFT algorithm which was discussed in the previous sections has been implemented in the following way (*psuedo* code is presented). There are many advantages and disadvantages of the algorithm that has been presented in the first chapter.

Algorithm 1: Mixed radix FFT Graph Flow (Simplified version)

input : Time domain inputs n in normal order

output: Frequency domain output k in bit reversed order as discussed earlier

```
1 Initialization with currentStage; nextStage; firstStage parameters;
2 for stages  $\leftarrow 1$  to no.ofstages do
3   if current stage == 2 then // Checks for current stage
4     Assign CurrentStageValue;
5     for index  $\leftarrow 1$  to firststage  $\times$  nextstage  $-$  nextstage  $+ 1$  do
6        $P = \text{Next stage} / \text{current stage}$ ;
7       for  $j \leftarrow 1$  to  $P$  do
8         Radix values  $\leftarrow$  Radix-2 calculations;
9         Twiddle Factors  $\leftarrow$  Twiddle factors multiplied;
10      end
11      increment index by nextstage;
12    end
13  end
14  Use the same loops and logic for other stages as well;
15  foreach element e for all the stages  $i$  do Same Algorithm but for respective radix unit;
16 end
```

4.1 Problems in implementation

[9] There are many disadvantages on analysing algorithm mentioned above on hardware. Following are few of them. Most importantly, they affect performance and speed of operation which is critical for all the digital application.

1. Complexity of the radix unit used.
2. Unified memory approach.
3. Necessity for pipelining the Hardware.

Since we are dealing with mixed radix implementations, radix units may range from 2,3,5,7 ... As the size of radix unit increases, the biggest disadvantage is the complexity of the hardware involved in building radix unit. But complexity is at the cost of increased processing speed. Hardware complexity used in building higher radix units can be reduced if we simplify the equations involved in processing the higher radix unit. Another alternative is to use the higher radix unit to process all the radices lower than the base radix unit. For example, simplified version of radix-25 unit is constructed and it is utilised for radices lower than 25 (2,3,5,7,13,17...25)

The biggest [12] disadvantage of the above algorithm is the location of elements in a same single memory unit which leads to memory contention. Inputs in time domain are stored in normal order for processing. If all the elements are contiguously stored in a single memory, accessing them simultaneously depends on the number of ports available in the memory for reading and writing at the same time. However, if we increase the number of ports available in the memory, it increases the cost of the SRAM memory unit. Therefore, using a single memory approach is not a good idea for a faster implementation of FFT on hardware. The algorithm that has been discussed above uses a unified memory scheme and thereby is not an ideal solution for a faster mixed radix implementation. Memory contentions is the major disadvantage.

Pipelining [9] hardware improves frequency of operation of a particular hardware. Since pipelining also has an effect on the area, it is very important to have a trade-off between area and speed of operation. If we increase the number of pipelining registers in a mixed radix FFT processor, it'll further increase area of entire hardware. Increasing the area through pipelining is not a great option, because mixed radix implementation consumes huge area. In applications where area is not critical, a similar version of hardware is preferred.

In mixed radix FFT's, there is a necessity to design a memory system where we can access elements simultaneously without any latency. The idea of accessing elements simultaneously will improve performance and speed of operation. Following section clearly gives an idea on how to design an efficient memory system that can improve speed of operation.

5 Mixed Radix Design Improvements

One of the earliest attempts to improve the speed of operation for a mixed radix applications came from the L G Jonhson's [9]paper on Conflict Free Memory Addressing for Dedicated FFT Hardware. A multi bank scheme has been developed for a fixed radix for high-speed implementations. In order to minimise the memory size, an in-place memory strategy is used, which means initial arrangement of the inputs into different banks is very important to avoid all the memory contentions associated with it in the subsequent stages of FFT implementation. This is specifically known as memory bank conflict free addressing scheme and helps in accessing data simultaneously into a radix-r unit in subsequent stages. The smaller radix units are often referred to as the butterflies, as they have a structure similar to that of the insect. In history, high speed transformations have been obtained by implementing pipelining in the hardware related to these butterfly units. For most of the higher throughput applications, pipelined architectures are used.

Pipelined architectures need $\log_r N$ processing elements, where N and r represent the length of FFT and the radix unit involved. One of the biggest down side of using pipelined architecture is that they consume relatively large area. In order to reduce the hardware size, we go for a memory based implementation. Memory based implementations are sequential circuits and require many computation cycles demanding higher running frequency for the entire hardware. Fixed radix butterfly units have a flow graph shown in the fig. 20. The biggest advantage of having a radix-r unit in a fixed radix system is the same unit with an additional control circuitry can be used in the entire implementation.

Consider a radix- r FFT butterfly unit. It has r inputs and r outputs to the butterfly, that are read and written in one pipeline cycle. Since there are r -inputs that are expected simultaneously, in order to avoid the memory bottleneck, memory is divided into separate banks and ensured that r -inputs and outputs are read/written at the same time. We can use an "in-place" strategy that has the ability to store the outputs from the butterfly in a permutation of those memory locations that will be used as inputs to the radix- r butterfly unit. The major advantage with this kind of architecture is that the speed requirement of an FFT application is readily met with all the memory modules.

5.1 In place strategy for Radix- r algorithm

FFT hardware that using the in-place strategy requires requires only one memory space for each of the point. [9] FFT hardware that uses the in-place strategy and two N -word memories can overlap the continuous flow FFT computations with the concurrent input and output.

Consider an example to illustrate the in-place strategy. A radix-4 unit butterfly needs to process 4 inputs at the same time and write into 4 different memory bank locations at the same time. In simple words, concurrent reads and write is the most important criteria for in-place memory operations. Fig. 2016 point example using radix-4 butterfly will clearly illustrate the banking scheme. The memory is partitioned into 4 banks and this way, we can eliminate all the memory conflicts that result while reading four elements and writing four elements at the same time.

Table shown below clearly shows two different columns, one for the bank and the other for address within the bank. These banks and address assignments are very unique and avoid all the memory that can potentially occur. Four inputs can be read from different banks and four inputs be written to the different banks for all the butterflies present in the FFT arrangement. One interesting application with the hardware is as soon as the first incoming symbol that is stored in the

5 Mixed Radix Design Improvements

corresponding memory bank and address is computed, it can be read out of the corresponding memory units and a second memory symbol is stored at the same memory location.

The general formula for Bank and address of a particular input is given as follows

$$x_{n-1}x_{n-2}x_{n-3}\dots x_0$$

$$Address = x[n-1]x[n-2]\dots x[3]x[2] \quad (5.1)$$

$$Bank = x[n-1] \oplus x[n-2]x[n-3] \oplus \dots \oplus x[1]x[0] \quad (5.2)$$

In the above formulae [13], $\oplus$ operator will be operating as a modulo 4 between corresponding bits. One of the biggest advantages of this in-place memory strategy is that once input symbol's FFT is computed, it is read out and the next symbol is stored at the same location concurrently.

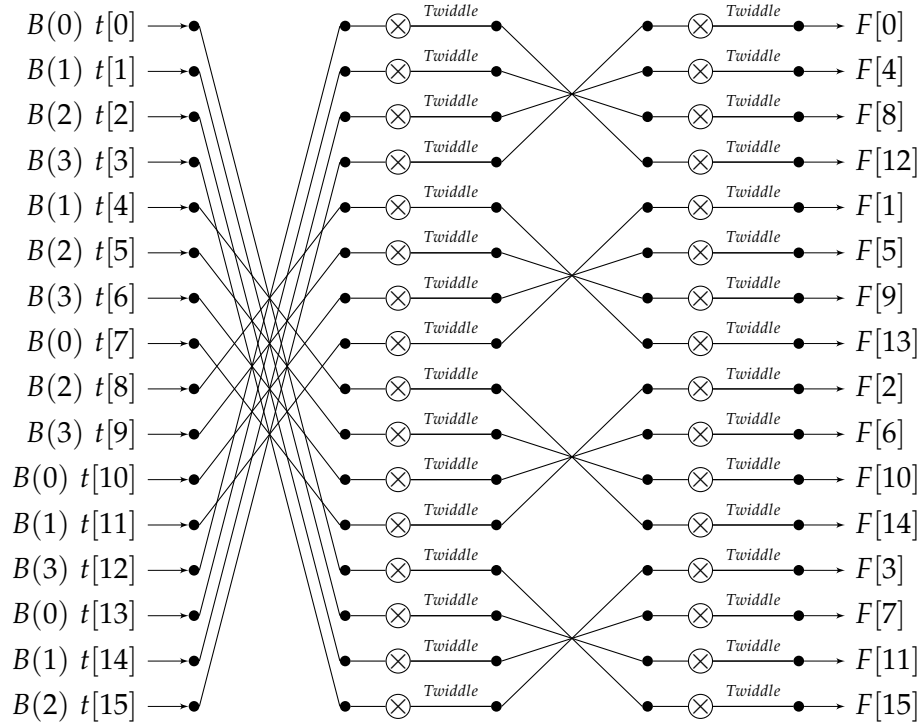
Examples with 16-point FFT will clearly illustrate application of the above formulae. Consider input order 2 = 0010, bank that is selected for this input would be $00 \oplus 10$, which is 2^{nd} bank. The address within the bank is given by 00. This is a simple calculation that has resulted from above formulae. Fig 20 clearly shows the bank and address information for all the input points and the output order as well.

Similarly, for others

Input	Bank	Address
2	2 = 10	0 = 00
3	3 = 11	0 = 00
7	0 = 00	1 = 01
..	..	..

The output order is different from the input order. Consider for a radix-4 algorithm, if $x_3x_2x_1x_0$ is the input order in the time domain, in the frequency domain output ordering is different and

Figure 20: Banking Scheme N = 16



given by $x_1x_0x_3x_2$. It'll be clear with an example, if the input order is 0001 (1) in the time domain, the output order would be 0100 (4) in the frequency domain. A simple switch network can be used as a hardware to generate such a reverse digit model in the frequency domain.

Another example using radix-4 for memory based implementations is to consider 64 point FFT application. Following is the bank and address for a corresponding input point in the time domain.

5 Mixed Radix Design Improvements

Input	Bank	Address
000001	1 = 01	0 = 00
000010	2 = 10	0 = 00
000011	3 = 11	0 = 00
000100	1 = 01	1 = 01
000101	2 = 10	1 = 01
000110	3 = 11	1 = 01
000111	0 = 00	1 = 01
001000	2 = 10	2 = 10
001001	3 = 11	2 = 10
..	..	..

We notice that all inputs to the radix-4 unit while processing elements in each of the stages are from distinct memory banks. In this way, we can avoid memory contentions while processing elements in the serial fashion. [13] But there is a dig disadvantage associated with the banking scheme that has been discussed earlier it works for even power of 2, which means the output order that has been generated in the Fig(ref) after performing an FFT can be used by the second symbol. To clearly explain this, after FFT computations, the computed symbol is read out and the next incoming symbol is stored at the same location concurrently.

Considering an odd power of 2, the banking scheme will not work for the even and odd symbols, which means that the output symbols after being read out, same index values cannot be stored as inputs for the second symbol FFT operations.

5.2 Mixed radix FFT schemes for odd power of 2

For an odd power of 2, method suggested above cannot be generalised because of the inherent conflict memory problems. There have been modifications suggested to the same algorithm to tackle these non-trivial cases.

5 Mixed Radix Design Improvements

Interchanging the storage locations of radix output will help in the resolving the conflicts. There will be change in the location of second and third output of each radix-4 unit in the 2nd Stage and a pair of radix 2 units in the 3rd Stage are also exchanged. After incorporating these changes, memory contentions are resolved and asymmetric form that has been resulted using the previous algorithm is converted to the symmetric form.

A more generalised version of changes that we discussed earlier can be put into a mathematical form as shown below. As discussed earlier, the most significant bits are exchanged with the least significant bits, and rest of the bits are reversed for the output order. Let the input order be represented by, $x_{n-1}x_{n-2}x_{n-3}x_{n-3}.....x_2x_1x_0$ and the output order will be represented by $x_1x_0x_2x_3.....x_{n-4}x_{n-2}x_{n-1}x_{n-2}$. This is the mathematical simplification that resulted from our modification.

The generalised formulae for bank and address in the modified case would be as follows:

$$Address = x[n-1]x[n-2].....x[3]x[2] \quad (5.3)$$

$$Bank = x[n-2] \oplus x[n-1]x[n-3] \oplus \oplus x[3]x[2] \oplus x[1]x[0] \quad (5.4)$$

Another example using Mixed radix FFT (4/2-radix) for memory based implementations is to consider 32 point FFT application. Following is the bank and address for a corresponding input point in the time domain.

5 Mixed Radix Design Improvements

Input	Bank	Address
0000001	1 = 01	0 = 00
0000010	2 = 10	0 = 00
0000011	3 = 11	0 = 00
0000100	1 = 01	1 = 01
0000101	2 = 10	1 = 01
0000110	3 = 11	1 = 01
0000111	0 = 00	1 = 01
0001000	2 = 10	2 = 10
0001001	3 = 11	2 = 10
0001010	1 = 00	2 = 10
0001011	1 = 01	2 = 10
0001100	3 = 11	3 = 11
0001101	0 = 00	3 = 11
0001110	1 = 01	3 = 11
..	..	..
1111111	2 = 10	31 = 11111

6 Prime Factorisation Algorithm (PFA) for Fast Fourier Transforms

6.1 Evolution of PFA

We have discussed about the generalised case N that can be decomposed into further to compute an FFT. For example, Decimation in Time (DIT), Decimation in frequency (DIF) which usually follows a divide and conquer approach. But, the general down side of this method is that for a given case of an N -point, N should be either a power of 2 or 4 or a mixture of two in the split radix case. However, we can pad with 0's in case given N -point is not a power of 2 or 4. The circuit complexity involved in padding 0's also increases with the size of FFT. Extra circuit complexity in padding, time required for padding and the nature of FFT has forced us to think in terms of mixed radix FFT, [7] that can handle radix unit other than 2 and 4. Single port memory approach creates memory bottlenecks and memory partitioning is necessary to compute FFT of each radix unit simultaneously.

The problem of memory contention can be further be simplified if we map a single dimension to n -dimensional space, where n depends on the maximum radix unit that has been used in the algorithm. Mapping will enable us to simplify our problem further and improve speed of operation and performance. The entire method is based on the fact that for a specific FFT, we pre-calculate certain things (to be discussed later) before hand and implement an "in-place" and "conflict free memory" policies. We are initially targeting for FFT's without any twiddle factors and in case

6 Prime Factorisation Algorithm (PFA) for Fast Fourier Transforms

there are any twiddle factors, they are implemented within the radix unit.

First attempt at this idea was given by Good's. Good's mapping on an original N -point set will transform it into small DFT's with the coprime lengths.

$$[x_{i+N_1k_2} | k_2 = 1, \dots, N_2 - 1] \text{ OR } [x_{i+N_2k_1} | k_1 = 1, \dots, N_1 - 1]$$

$$i = [k_1N_2 + k_2N_1] \bmod N \quad \text{where } k_1 = 1, \dots, N_1 - 1; k_2 = 1, \dots, N_2 - 1; N = N_1N_2 \quad (6.1)$$

As shown in the figure below, there will be 1-1 mapping from each element in the single dimension to all the other elements in the multi dimensions, only if N_1 and N_2 are coprimes.

$$0 \quad 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6 \quad 7 \quad 8 \quad 9 \quad 10 \quad 11$$

Using Good's mapping can be transformed as follows

$$\begin{array}{cccc} 0 & 11 & 2 & 5 \\ 4 & 6 & 9 & 8 \\ 10 & 1 & 3 & 7 \end{array}$$

This idea can be further extended for mixed radix FFT's. Mapping from a single dimension to a two dimensional space can be interpreted physically as mapping from a single memory system to two distinct memory banks. Similarly, we can extend the idea further into an n -dimensional space that is equivalent to having n -different banks in hardware. Our next objective is to find a unique 1-1 mapping from single dimensional unified memory to a multi-bank memory system, which is nothing but multi-dimensional space in Mathematical terms.

6 Prime Factorisation Algorithm (PFA) for Fast Fourier Transforms

Using Chinese Remainder Theorem [3] we get another transformation that can facilitate a multi-dimensional transform. Chinese remainder states that when the moduli of system of liner congruences are pairwise relatively prime, there exist a unique solution of the system modulo the product of moduli. Let $k_1, k_2, \dots, k_r$ be positive interferences such that $k_i \perp k_j$ for all $i \neq j$

Then the system [19] of linear congruences:

$$x \equiv [c_1] \text{mod } k_1$$

$$x \equiv [c_2] \text{mod } k_2$$

$$x \equiv [c_3] \text{mod } k_3$$

..

$$x \equiv [c_r] \text{mod } k_r$$

has a solution which is unique modulo $\prod_{i=1}^r k_i$

x will satisfy the following equation

$$x \equiv [c_1 l_1 \frac{N}{k_1} + \dots + c_r l_r \frac{N}{k_r}] \text{ (mod } N) \quad (6.2)$$

where $N = k_1 k_2 k_3, \dots, k_r$

l_i 's are determined from $l_i \frac{N}{k_i} \equiv 1 \text{ (mod } n_i)$

In simple terms or in the context that is relevant to the FFT's, the Chinese remainder theorem states that, if we know residue of number k modulo two co-prime numbers N_1 and N_2 i.e., $(k \text{ mod } N_1)$ is a known value and $(k \text{ mod } N_2)$ is also a known value. We can construct $k \text{ mod } (N_1 N_2)$.

6 Prime Factorisation Algorithm (PFA) for Fast Fourier Transforms

$$k \bmod N_1 N_2 = [N_1 q_1 k_2 + N_2 q_2 k_1] \bmod N \quad (6.3)$$

From 6.2 that has been explained earlier:

q_1 is multiplicative inverse $= N_1^{-1} \bmod N_2$

q_2 is multiplicative inverse $= N_2^{-1} \bmod N_1$

0 1 2 3 4 5 6 7 8 9 10 11

Using Chinese remainder theorem it can be transformed as follows

0 6 2 10
4 1 3 9
11 5 7 8

Applying all the techniques that have been discussed so far, will result in the following simplifications on a DFT

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{nk} \text{ where } k = 0, \dots, N-1, \quad (6.4)$$

Using the Chinese remainder theorem and the Good's mapping criteria we are substituting 6.3 and 6.1 in the actual FFT equation 6.5

$$X[N_1 l_1 k_2 + N_2 l_2 k_1] = \sum_{n_1=0}^{N_1-1} \sum_{n_2=0}^{N_2-1} x[n_1 N_2 + n_2 N_1] W_N^{(n_1 N_2 + n_2 N_1)(N_1 l_1 k_2 + N_2 l_2 k_1)} \quad (6.5)$$

6 Prime Factorisation Algorithm (PFA) for Fast Fourier Transforms

Using $W_N^{N_2 c_1} = W_{N_1}^{c_1}$; $W_N^{N_1 c_2} = W_{N_2}^{c_2}$, we can further simplify

$$X[k_1, k_2] = X[N_1 l_1 k_2 + N_2 l_2 k_1] \text{ and } x[n_1, n_2] = x[n_1 N_2 + n_2 N_1]$$

Using all the above substitutions and simplifications,

$$X[k_1, k_2] = \sum_{n_1=0}^{N_1-1} \sum_{n_2=0}^{N_2-1} x[n_1, n_2] W_{N_1}^{n_1 k_1} W_{N_2}^{n_2 k_2} \quad (6.6)$$

We can clearly see that there are no twiddle factors in 6.6, for a decomposition that is $N = N_1 N_2$, where N_1 and N_2 are co-primes to each other. The advantage of this method is it is simplifying the entire procedure involved in processing an FFT, but the disadvantage is decompositions should be relatively prime to each other. This way, we are eliminating the effect of twiddle factors. However, all the decompositions need not be co-prime to each other.

6.2 Prime Factor Algorithm and its implications

Application of [11] [17] Prime factor Algorithm is critical for FFT's as we've discussed earlier that the kind of mapping from a single-dimension to multi-dimensional space decides the speed of operation and its performance. C. Sidney Burrus has generalised the transformation for many cases and forms basis for my thesis.

The transformations from one-dimension to multi-dimensions gives theoretical and computational advantage in many cases. Basis for all these transformation would come from the congruence theory. Again, our objective is to $x(n)$ to $x(n_1, n_2)$ for an N -point transform, where $N = N_1 N_2$. Also, the range of n would be from $0, \dots, N-1$. On the other hand, range for n_1 is $0, \dots, N_1 - 1$ and for n_2 is $0, \dots, N_2 - 1$. Shown below is the unique mapping for all possible decompositions of

N.

A simple mapping a one dimension to two dimension is to consider a linear case. The mapping ignores case where the transformation depends on higher order terms.

$$n = [K_1n_1 + K_2n_2] \bmod N \quad (6.7)$$

Since the Eq. 6.7 mapping is evaluated on modulo N it is cyclic in N. For the above mapping, we need to determine K_1 and K_2 for which there exist a unique mapping from each element in the n domain to (n_1, n_2) domain. We can use number theory and congruence properties that will help us to find (K_1, K_2) .

To find out (K_1, K_2) , we need to handle two cases:

1. N_1, N_2 are relatively prime to each other.
2. N_1, N_2 have a common multiple.

Theorem clearly states that the necessary and final conditions for the mapping to exist is:

When N_1 and N_2 are relatively prime:

1. $(K_1 = \alpha N_2)$ and $(K_2 \neq \beta N_1)$ and $(\alpha, N_1) ; (K_2, N_2)$ are coprimes to each other.
2. $(K_1 \neq \alpha N_2)$ and $(K_2 = \beta N_1)$ and $(K_1, N_1) ; (\beta, N_2)$ are coprimes to each other.
3. $(K_1 = \alpha N_2)$ and $(K_2 = \beta N_1)$ and $(\alpha, N_1) ; (\beta, N_2)$ are coprimes to each other.

6 Prime Factorisation Algorithm (PFA) for Fast Fourier Transforms

When N_1 and N_2 are not relatively prime:

1. $(K_1 = \alpha N_2)$ and $(K_2 = \beta N_1)$ and $(\alpha, N_1) ; (K_2, N_2)$ are coprimes to each other.
2. $(K_1 \neq \alpha N_2)$ and $(K_2 \neq \beta N_1)$ and $(K_1, N_1) ; (\beta, N_2)$ are coprimes to each other.

Let us consider examples to illustrate these results. Let $N = 2 \times 5$, $K_1 = c_1 N_2$, but let us consider $c_1 = 1$, then $K_1 = N_2$. $K_1 = 5$, $K_2 = 1, 2, 3, 4, 6, 7, 8, 9, 11 \dots K_2 \neq 5, 10, 15, 20$, since they are multiples of N_2 . From the above conditions, we have to choose K_2 , such that they are cyclic in K_1, K_2 , $K_2 = 2, 4, 6, 8, 10 \dots$ etc. Any number from the set can be chosen as they satisfy the condition that $K_1 = \alpha N_2$ and $K_2 = \beta N_2$

If we consider K_1 to be a non-multiple of N_2 , for example, $K_1 = 3$, K_2 should definitely be a multiple of N_1 .

Consider another exclusive case where factors after decomposition of N need are not coprime to each other. For example, $N = 10 \times 4$, let $K_1 = 4$. K_2 can take values from $1, 5, 7, 17 \dots$ etc applying all the above equations.

There can be many improvements in the mapping that can affect the complexity of the algorithm and on the other hand even simplify the implementation of a particular application on Hardware.

We can use residue theorem and Chinese Remainder Theorem to further achieve unique one-one mapping scheme. Since the new (K_1, K_2) variables are supposed to be cyclic, we can choose the following combination:

1. $\alpha = N_2^{-1} \bmod N_1$
2. $\beta = N_1^{-1} \bmod N_2$

From Eq 6.7, above two equations reduces to CRT problem.

6.2.1 PFA for FFT applications

Prime factor Algorithm [17] establishes a unique mapping from a single dimension to multiple dimensions using the Chinese remainder theorem as shown above. Its application in FFT reduces complexity of the algorithm and number of computations involved. [16] The advantage of applying PFA to FFT is, we can use the periodic properties of the FFT to further simplify the complexity of Algorithm.

Consider the definition of FFT, with exponent evaluated to the modulo of N satisfying the periodic property mentioned before.

$$X[k] = \sum_{n=0}^{N-1} x[n] W_N^{(nk) \bmod N} \text{ where } k = 0, \dots, N-1, \quad (6.8)$$

Modulo operator in the exponent of Eq. 6.10 takes care of the periodicity.

As discussed earlier, linear mapping for both the indices result in following equations.

$$\begin{aligned} n &= [K_1 n_1 + K_2 n_2] \bmod N \\ k &= [K_3 k_1 + K_4 k_2] \bmod N \end{aligned}$$

Substituting these transformations into our main equation 6.10 results in the following:

$$X(K_3 k_1 + K_4 k_2) = \sum_{n_1=0}^{N_1-1} \sum_{n_2=0}^{N_2-1} x(K_1 n_1 + K_2 n_2) W_N^{(K_1 n_1 + K_2 n_2)(K_3 k_1 + K_4 k_2) \bmod N} \quad (6.9)$$

6 Prime Factorisation Algorithm (PFA) for Fast Fourier Transforms

Equation 6.9 in two dimensions will be as follows:

$$X[k_1, k_2] = \sum_{n_1=0}^{N_1-1} \sum_{n_2=0}^{N_2-1} x[n_1, n_2] W_N^{(K_1 n_1 + K_2 n_2)(K_3 k_1 + K_4 k_2) \bmod N} \quad (6.10)$$

$$W_N^{nk} = W_N^{K_1 K_3 n_1 k_1 + K_2 K_3 n_2 k_1 + K_1 K_4 n_1 k_2 + K_2 K_4 n_2 k_2} \quad (6.11)$$

In order to simplify the huge equation, we have the freedom to choose K_1, K_2, K_3, K_4 .

At this stage we can divide [8] the entire problem into two cases, first one (N_1, N_2) have no common multiples and the other case is where (N_1, N_2) have a common multiple.

When N_1, N_2 do not have a common multiple. Choose

Which simplifies the arrangement, as

$$W_N^{n_1 k_2 K_1 K_4} = 1 \quad W_N^{n_2 k_1 K_2 K_3} = 1$$

Simplified form of the FFT when (N_1, N_2) do not have common factor would be:

$$X[k_1, k_2] = \sum_{n_1=0}^{N_1-1} \left[\sum_{n_2=0}^{N_2-1} x[n_1, n_2] W_{N_2}^{c_2 c_4 N_1 n_2 k_2} \right] W_N^{c_1 c_3 N_2 n_1 k_1} \quad (6.12)$$

From the residue theorem [19] and Chinese remainder theorem eq6.2, we can further simplify equations as follows:

6 Prime Factorisation Algorithm (PFA) for Fast Fourier Transforms

$$\begin{aligned} c_1 &= c_2 = 1 \\ c_4 &= N_1^{-1} \bmod N_2 \\ c_3 &= N_1^{-1} \bmod N_2 \end{aligned}$$

Above equation results in the following simplification:

$$X[k_1, k_2] = \sum_{n_1=0}^{N_1-1} \left[\sum_{n_2=0}^{N_2-1} x[n_1, n_2] W_{N_2}^{n_2 k_2} \right] W_N^{n_1 k_1} \quad (6.13)$$

Eq 6.13 presents the final form in which inner expression (n_2) is independent of the outer expression (n_1)

Let us consider another [17] case where N_1, N_2 are not relatively prime to each other. In this case, we cannot use the simplification $K_1 = c_1 N_2$ and $K_4 = c_4 N_2$, since we cannot find K_1, K_4 that satisfy these conditions

$$W_N^{nk} = W_N^{K_1 K_3 n_1 k_1 + K_2 K_3 n_2 k_1 + K_1 K_4 n_1 k_2 + K_2 K_4 n_2 k_2 \bmod N} \quad (6.14)$$

Since we cannot use substitution for K_1 and K_4 , so using the earlier same substitution for $K_2 = c_2 N_1$ and $K_3 = c_3 N_2$. After solving the Eq 6.14, we end up in the following equation:

$$X[k_1, k_2] = \sum_{n_1=0}^{N_1-1} \left[\sum_{n_2=0}^{N_2-1} x[n_1, n_2] W_{N_2}^{c_2 K_4 n_2 k_2} \right] \left[W_{N_2}^{K_1 K_4 n_1 k_2} \right] W_N^{K_1 c_3 n_1 k_1} \quad (6.15)$$

In the Eq 6.15, if we substitute $K_1 K_4$ product as 1, then we can form a simple twiddle factor that multiplies with summation output from inner product. Either it can be grouped with inner product n_2 , or it [16] can grouped with the outer product n_1 , giving us the flexibility to handle different

6 Prime Factorisation Algorithm (PFA) for Fast Fourier Transforms

cases.

$c_1 = c_3 = K_1 = K_4 = 1$ gives us the final transformations as follows

$$n = n_1 + N_1 n_2 \bmod N$$

$$k = N_2 k_1 + k_2 \bmod N$$

Final equation would be:

$$X[k_1, k_2] = \sum_{n_1=0}^{N_1-1} \left[\sum_{n_2=0}^{N_2-1} x[n_1, n_2] W_{N_2}^{n_2 k_2} \right] \left[W_N^{n_1 k_2} \right] W_N^{n_1 k_1} \quad (6.16)$$

$$\left[W_N^{n_1 k_2} \right] \quad (6.17)$$

Eq 6.16 clearly shows that there is an extra twiddle factor to handle. Eq 6.17 in bigger square bracket in Eq 6.16 specifies that it can be included in the inner product or the outer product.

Let us summarise these transformation for now,

If we decompose N-point FFT into N_1 and N_2 . After decomposition $N = N_1 N_2$, we transform n and k into a two dimensional domain $(n_1, n_2), (k_1, k_2)$ in the following way:

$$\begin{aligned} n &= [N_2 n_1 + K_1 n_2] \bmod N \quad n_1, k_1 = 0, 1, \dots, N_1 - 1, \\ k &= [K_2 k_1 + N_1 k_2] \bmod N \quad n_2, k_2 = 0, 1, \dots, N_2 - 1, \end{aligned} \quad (6.18)$$

6 Prime Factorisation Algorithm (PFA) for Fast Fourier Transforms

Case 1) If N_1 and N_2 are relatively prime, choose K_1 and K_2 that satisfy following equations

$$K_1 = a_1 N_1 \text{ and } K_1 = a_2 N_2 + 1$$

$$K_2 = a_3 N_2 \text{ and } K_2 = a_4 N_1 + 1$$

$$\begin{aligned} X[k_1, k_2] &= \sum_{n_2=0}^{N_2-1} \left[\sum_{n_1=0}^{N_1-1} x[n_1, n_2] W_{N_1}^{n_1 k_1} \right] W_{N_2}^{n_2 k_2} \\ &= \sum_{n_2=0}^{N_2-1} \left[y(k_1, n_2) \right] W_{N_2}^{n_2 k_2} \end{aligned} \quad (6.19)$$

Case 2) If N_1 and N_2 are not relatively prime, choose K_1 and K_2 as follows

$$K_1 = 1$$

$$K_2 = 1$$

$$\begin{aligned} X[k_1, k_2] &= \sum_{n_1=0}^{N_2-1} W_N^{n_2 k_1} \left[\sum_{n_1=0}^{N_1-1} x[n_1, n_2] W_{N_1}^{n_1 k_1} \right] W_{N_2}^{n_2 k_2} \\ &= \sum_{n_1=0}^{N_2-1} W_N^{n_2 k_1} \left[y(k_1, n_2) \right] W_{N_2}^{n_2 k_2} \end{aligned} \quad (6.20)$$

We can extend the idea to n-dimensions instead of 2-dimensions

6.2.2 More than two dimensional transformation using PFA on DFT's

Consider $N = N_1 N_2 N_3$, we can extend two dimensional idea to three dimensions by partitioning as follows,

$$N = N_1(N_2 N_3)$$

$$\begin{aligned} n &= [N_2 N_3 n_1 + K_1 \bar{n}_2] \bmod N \\ k &= [K_2 k_1 + N_1 \bar{k}_2] \bmod N \end{aligned} \tag{6.21}$$

$$K_1 = c_1 N_1 \text{ and } K_1 = c_2 N_2 N_3 + 1$$

$$K_2 = c_3 N_2 N_3 \text{ and } K_2 = c_4 N_1 + 1$$

Consider $N_2 N_3$ for further decomposition as a *new* N

$$\begin{aligned} \bar{n}_2 &= [N_3 n_2 + K_3 n_3] \bmod N_2 N_3 \\ \bar{k}_2 &= [K_4 k_2 + N_2 k_3] \bmod N_2 N_3 \end{aligned} \tag{6.22}$$

$$K_3 = c_5 N_2 \text{ and } K_3 = c_6 N_3 + 1$$

$$K_4 = c_7 N_3 \text{ and } K_4 = c_8 N_2 + 1$$

6.2.3 Examples of PFA on DFT's

Let us consider three cases to illustrate how decomposition actually works on natural numbers

6 Prime Factorisation Algorithm (PFA) for Fast Fourier Transforms

1. $N = 2 \times 3$, where $N_1 = 2$, $N_2 = 3$ and they both are relatively prime to each other.

$$n = [3n_1 + K_1n_2] \bmod N \quad n_1, k_1 = 0, 1,$$

$$k = [K_2k_1 + 2k_2] \bmod N \quad n_2, k_2 = 0, 1, 2$$

$$K_1 = 2a_1 \text{ and } K_1 = 3a_2 + 1$$

$$K_2 = 3a_3 \text{ and } K_2 = 2a_4 + 1$$

From the above equations we can decide that:

$$a_1 = 2$$

$$a_2 = 1$$

$$a_3 = 1$$

$$a_4 = 1$$

Final transformation would be:

$$\begin{aligned} n &= [3n_1 + 4n_2] \bmod N \\ k &= [3k_1 + 2k_2] \bmod N \end{aligned} \tag{6.23}$$

2. $N = 2 \times 4$, where $N_1 = 2$, $N_2 = 4$ and they both are NOT relatively prime to each other.

$$n = [4n_1 + n_2] \bmod N \quad n_1, k_1 = 0, 1$$

$$k = [k_1 + 2k_2] \bmod N \quad n_2, k_2 = 0, 1, 2, 3$$

Since N_1 and N_2 are NOT relatively prime to each other, K_1 and $K_2 = 1$

6 Prime Factorisation Algorithm (PFA) for Fast Fourier Transforms

Final transformation would be;

$$\begin{aligned} n &= [4n_1 + n_2] \bmod N \\ k &= [k_1 + 2k_2] \bmod N \end{aligned} \tag{6.24}$$

3. Third case would deal exclusively with a transformation from one dimension to a three dimensions.

Consider $N = 2 \times 3 \times 2$ in $N_1 \times N_2 \times N_3$ order Using the Eq. 6.23 and Eq. 6.22 applying first level decomposition (2×6):

$$\begin{aligned} n &= [6n_1 + 1 \times \bar{n}_2] \bmod 12 \quad n_1, k_1 = 0, 1 \\ k &= [1 \times k_1 + 2\bar{k}_2] \bmod 12 \quad n_2, k_2 = 0, 1, 2, 3, 4, 5 \end{aligned}$$

Further decomposition on $\bar{n}_2$ and $\bar{k}_2$ results in (2×3) :

$$\begin{aligned} \bar{n}_2 &= [2n_2 + K_1n_3] \bmod 6 \\ \bar{k}_2 &= [K_2k_2 + 3k_3] \bmod 6 \end{aligned}$$

$$K_1 = c_1 \times 3 = c_2 \times 2 + 1$$

$$K_2 = c_3 \times 2 = c_4 \times 3 + 1$$

$$\implies K_1 = 3 \text{ and } K_2 = 4$$

Final substitution leads us to,

$\implies$

$$n = (6n_1 + ((2n_2 + 3n_3) \bmod 6)) \bmod 12 = (6n_1 + 2n_2 + 3n_3) \bmod 12$$

$$k = (k_1 + 2((4k_1 + 3k_2) \bmod 6)) \bmod 12 = (k_1 + 8k_1 + 6k_2) \bmod 12$$

6.3 Banking scheme for conflict free processing

In the previous sections, we discussed about conflict-free processing for fixed radix system. However, there is a problem while handling mixed radix FFT's. [7] There is no scheme that maps elements from the single memory to n - different banks. The basis for mapping is to enable hardware to data read and write in a single cycle. For example consider $2 \times 3 \times 5$, as discussed in the naive mixed radix implementation shown earlier, first stage will be processed by radix-2 unit, second stage will be processed by radix-3 and the third stage will be processed by radix-5 unit. Since the maximum radix unit in the example given is 5, we need five inputs simultaneously, so that we can process them at once without waiting for other inputs. There is a need to come up with a scheme, where we can distribute elements into 5 memory banks so that they are conflict free in every stage.

Consider $N = N_1 \times N_2 \times N_3 \times N_4 \dots N_n$. It has N -stages and at each stage i we are computing using radix- N_i unit (no of radix- N_i units will be $\prod_{r=1}^{r=N} r \neq i$). We can define a transformation from one-dimension to n -dimensions as shown below using the PFA.

$$n = [N_2 N_3 \dots N_n n_1 + K_1 \bar{n}_2] \bmod N$$

$$k = [K_2 k_1 + N_1 \bar{k}_2] \bmod N$$

$$\bar{n}_2 = [N_3 N_4 \dots N_n n_2 + K_3 \bar{n}_3] \bmod N_2 N_3 \dots N_n$$

$$\bar{k}_2 = [K_4 k_2 + N_2 \bar{k}_3] \bmod N_2 N_3 \dots N_n$$

....

6 Prime Factorisation Algorithm (PFA) for Fast Fourier Transforms

In the first stage we need a mapping scheme such that all the N_1 points (because we are using radix - N_1 , that has N_1 inputs in the first stage) are in different memory banks (N_{max} in total). In the first stage, input data $x[n_1, \bar{n}_2]$ where n_1 ranges from $0, \dots, N_1 - 1$. For each distinct value of n_1 , $\bar{n}_2$ is same. In short, every distinct input from individual radix unit should be present in an different bank so that all elements can be processed simultaneously. Define a mapping shown below ('c' is a constant offset):

$$bank = [n_1 + c] \bmod N_{max} \quad (6.25)$$

In the second n_2 ranges from $0, \dots, N_2 - 1$. Again, we can define a mapping for the points to be present in different memory bank as follows.

$$bank = [n_2 + c] \bmod N_{max} \quad (6.26)$$

Similarly, for the n^{th} , mapping will be as follows,

$$bank = [n_n + c] \bmod N_{max} \quad (6.27)$$

Combining all the above equations 6.25 ; 6.26 ; 6.27 we can define a mapping scheme that is unique and can eliminate memory conflicts in all the stages as follows:

$$bank = [n_1 + n_2 + n_3 + \dots + n_n] \bmod N_{max} \quad (6.28)$$

Once bank is selected, address mapping [13] within the bank can be anything. for simplicity let us use the following address mapping with each bank

$$address = [c_1n_1 + c_2n_2 + c_3n_3 + + c_n n_n](except\ n_{max}\ combination\ not\ included) \quad (6.29)$$

Eq 6.28 6.29 gives us the final bank and address information for all inputs.

6.3.1 Examples of Mixed Radix FFT's using Banking schemes and PFA

We can consider simple examples to the complex ones as below.

1. $N = 10$ can be decomposed into 2×5 with $N_1=2$ and $N_2=5$. After transforming these into two dimensions as discussed above, we end up with following:

$$n = [5n_1 + 6n_2] \bmod 10$$

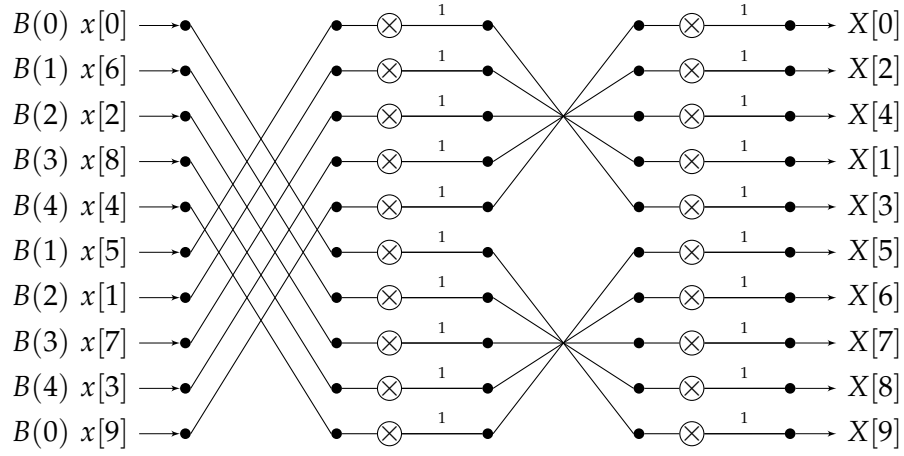
$$k = [5k_1 + 2k_2] \bmod 10$$

$$X[k_1, k_2] = \sum_{n_2=0}^4 \left[\sum_{n_1=0}^1 x[n_1, n_2] W_2^{n_1 k_1} \right] W_5^{n_2 k_2}$$

$$\text{bank} = [n_1 + n_2] \bmod 10$$

In the first stage, we are varying n_2 from $0 \rightarrow 4$ for all values of n_1 . In the second stage, we are varying n_1 from $0 \rightarrow 1$ for all values of n_2 . For example, all 10 values in the first stage are formed with (n_1, n_2) pairs sequence as $(0,0),(0,1), (0,2),(0,3),(0,4),(1,0),(1,1),(1,2),(1,3),(1,4)$. In the second stage, their sequence would be $(0,0),(1,0),(0,1),(1,1)(0,2)(1,2)(0,3)(1,3)(0,4)(1,4)$. The same pattern repeats for all the following example, so use the same idea for all.

Figure 21: Factors are coprimes to each other N = 10



Following Fig 21 clearly illustrates banking scheme for 10 point in place conflict free memory access schemes.

2. $N = 12$ can be decomposed into 2×6 with $N_1=2$ and $N_2=6$. After transforming these into two dimensions, we end up with following:

$$n = [6n_1 + n_2] \bmod 12$$

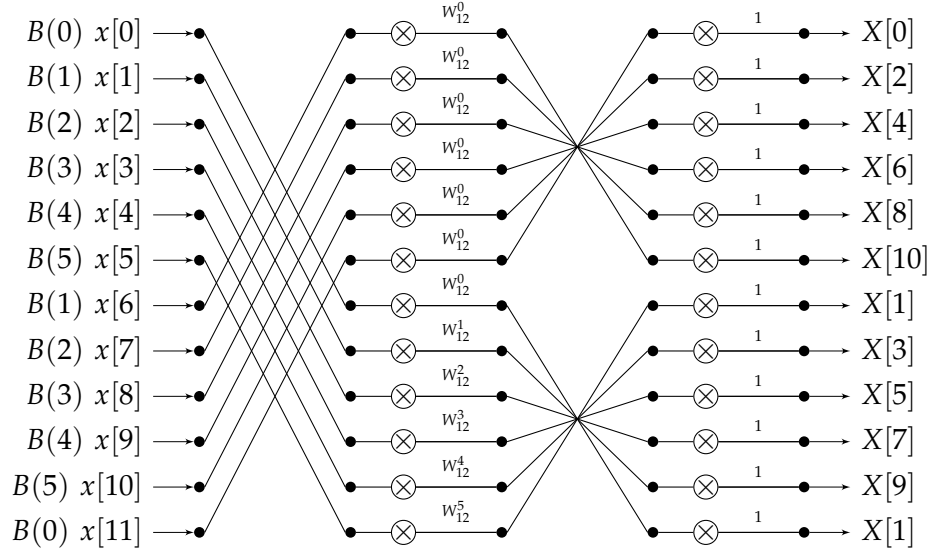
$$k = [k_1 + 2k_2] \bmod 12$$

$$X[k_1, k_2] = \sum_{n_1=0}^5 W_{12}^{n_2 k_1} \left[\sum_{n_1=0}^1 x[n_1, n_2] W_2^{n_1 k_1} \right] W_6^{n_2 k_2}$$

$$\text{bank} = [n_1 + n_2] \bmod 12$$

Following Fig 22 clearly illustrates banking scheme for 12 point in place conflict free memory access schemes.

Figure 22: Factors are NOT coprimes to each other N = 12



3. $N = 12$ can be decomposed into $2 \times 3 \times 2$ with $N_1=2$ and $N_2=3$ and $N_2=2$. After transforming these into three dimensions, we end up with following:

$$n = [6n_1 + 2n_2 + 3n_3] \bmod 12$$

$$k = [k_1 + 8k_2 + 6k_3] \bmod 12$$

$$\begin{aligned} X[k_1, k_2, k_3] &= W_{12}^{((6n_1+2n_2+3n_3)(k_1+8k_2+6k_3)) \bmod 12} \\ &= W_{12}^{(6n_1k_1+24n_1k_2+36n_1k_3+2n_2k_1+16n_2k_2+12n_2k_3+3n_3k_1+24n_3k_2+18n_3k_3) \bmod 12} \end{aligned}$$

$$X[k_1, k_2, k_3] = W_{12}^{((6n_1+2n_2+3n_3)(k_1+8k_2+6k_3)) \bmod 12}$$

$$= W_{12}^{(6n_1k_1+24n_1k_2+36n_1k_3+2n_2k_1+16n_2k_2+12n_2k_3+3n_3k_1+24n_3k_2+18n_3k_3) \bmod 12}$$

6 Prime Factorisation Algorithm (PFA) for Fast Fourier Transforms

With the following simplification

$$W_{12}^{(48n_1k_2+36n_1k_3+12n_2k_3+24n_3k_2)} = 1$$

$$W_{12}^{16n_2k_2 \bmod 12} = W_{12}^{4n_2k_2}$$

$$W_{12}^{18n_3k_3 \bmod 12} = W_{12}^{6n_3k_3}$$

Simplified version

$$W_{12}^{(6n_1k_1+2n_2k_1+4n_2k_2+3n_3k_1+6n_3k_3)}$$

Simplifying each individual terms to get a generic order

$$W_2^{n_1k_1} = W_{12}^{6n_1k_1}$$

$$W_3^{n_2k_2} = W_{12}^{4n_2k_2}$$

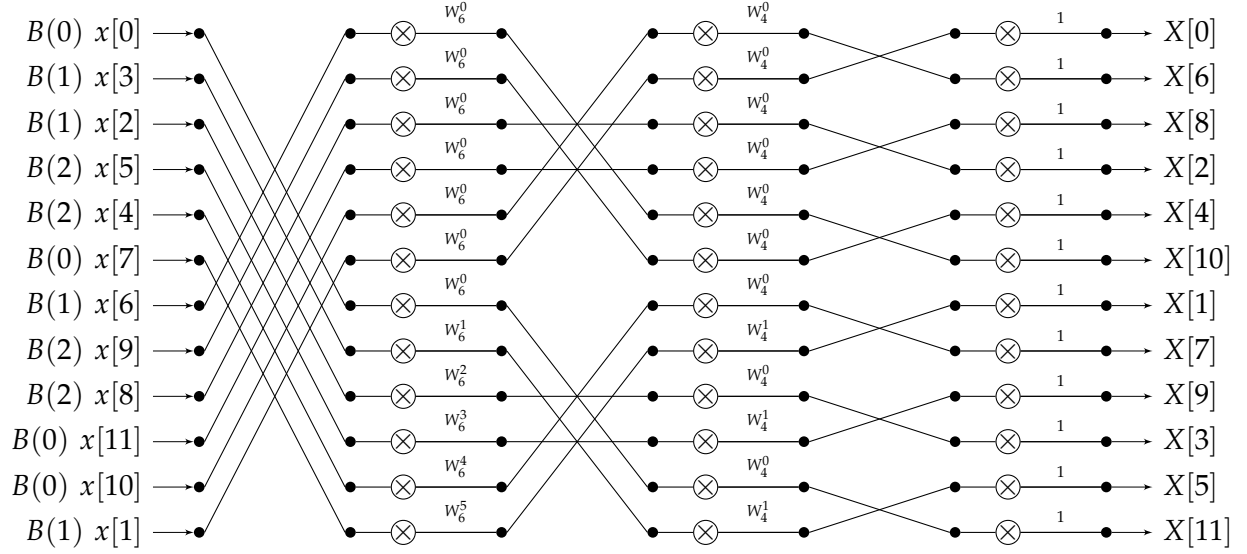
$$W_2^{n_3k_3} = W_{12}^{6n_3k_3}$$

$$W_6^{n_2k_1} = W_{12}^{2n_1k_1}$$

$$W_4^{n_3k_1} = W_{12}^{3n_3k_1}$$

Final equation would be,

Figure 23: Simple three dimensional transformation N = 12



$$X(k_1, k_2, k_3) = \sum_{n_3=0}^1 W_4^{n_3 k_1} \left[\sum_{n_2=0}^2 W_6^{n_2 k_1} \left[\sum_{n_1=0}^1 x(n_1, n_2, n_3) W_2^{n_1 k_1} \right] W_3^{n_2 k_2} \right] W_2^{n_3 k_3}$$

$$\text{bank} = [n_1 + n_2 + n_3] \bmod 12$$

Fig 23 clearly illustrates banking scheme for 12 point in place conflict free memory access schemes.

4. $N = 24$ can be decomposed into $2 \times 2 \times 2 \times 3$ with $N_1=2$ and $N_2=2$ and $N_2=2$ $N_4=3$. After transforming these into four dimensions, we end up with following:

$$n = [12n_1 + 6n_2 + 3n_3 + 4n_4] \bmod 24$$

$$k = (k_1 + 2k_2 + 12k_3 + 8k_4) \bmod 24$$

$$X[k_1, k_2, k_3, k_4] = W_{24}^{((12n_1+6n_2+3n_3+4n_4)(k_1+2k_2+12k_3+8k_4)) \bmod 60}$$

6 Prime Factorisation Algorithm (PFA) for Fast Fourier Transforms

Using the following simplification

$$W_{24}^{24c_1} = 1$$

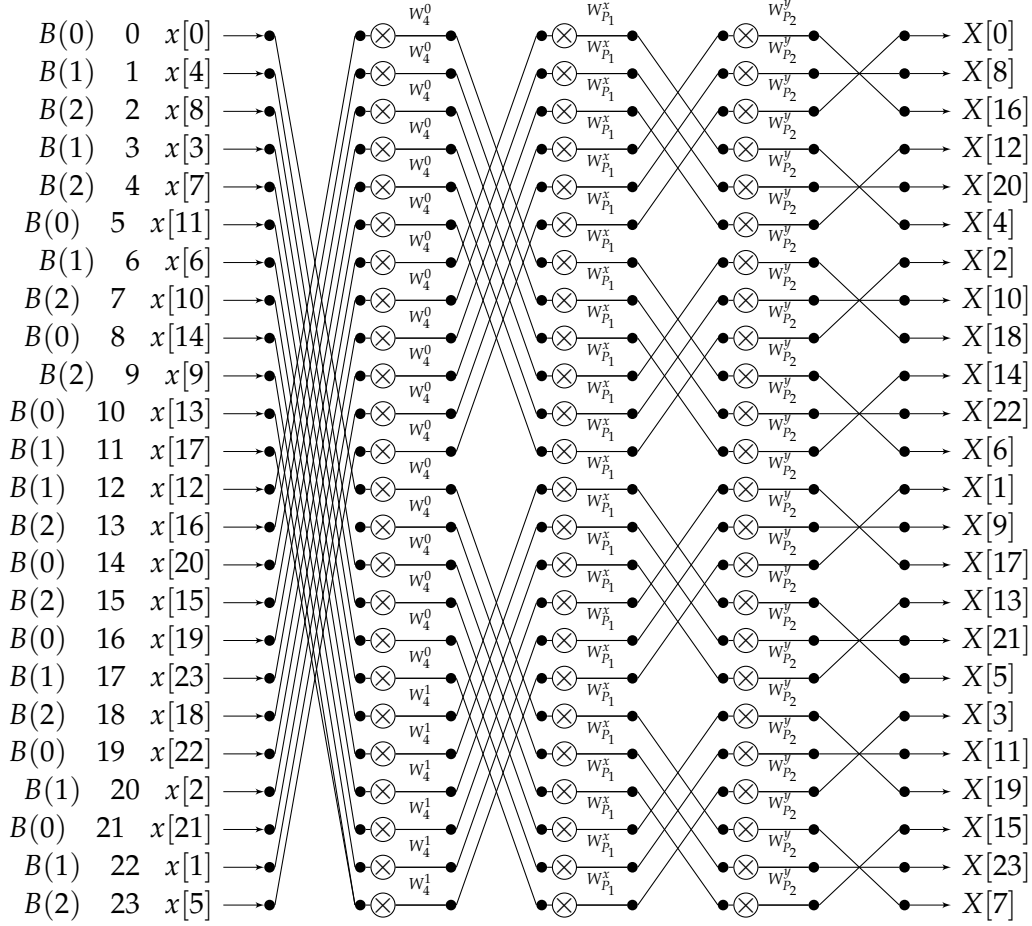
$$W_{24}^{abc \bmod 24} = W_{24}^{a \bmod 24b \bmod 24c \bmod 24}$$

Final equation would be,

$$X(k_1, k_2, k_3, k_4) = \sum_{n_4=0}^2 W_6^{n_4 k_1} W_3^{n_4 k_2} \left[\sum_{n_3=0}^1 W_4^{n_3 k_2} W_8^{n_3 k_1} \left[\sum_{n_2=0}^1 W_4^{n_2 k_1} \left[\sum_{n_1=0}^1 x(n_1, n_2, n_3, n_4) W_2^{n_1 k_1} \right] \right. \right. \\ \left. \left. W_2^{n_2 k_2} \right] W_2^{n_3 k_3} \right] W_3^{n_4 k_4}$$

Fig 24 clearly illustrates banking scheme for 24 point in place conflict free memory access schemes.

Figure 24: Three dimensional transformation with complex twiddle factors $N = 24$



7 Algorithm for in-place mixed radix FFT scheme using PFA

Pseudo code of the Algorithm for in-place conflict free processing in mixed radix cases has been shown below. Total number of stages in our N-point implementation decides number of *for* loops in the algorithm. Pseudo code is just to illustrate the idea, but we use a Finite state machine to implement the idea on Hardware. Hardware model will soon follow the pseudo code. As discussed before, each stage is associated with a variable n_i , and this variable is incremented till it reaches the maximum limit.

Every radix- r unit in each stage will have r inputs coming from different banks such that there is no memory contention. We are successful in mapping single dimension array into multiple dimensions and store them into different banks. If we are able to locate first point in the radix- r unit, we should be able to successfully locate other points without even calculating the bank number. We increment bank index by one, and find the next point, similarly navigate through all the other points until radix- r unit has all its inputs ready for computation.

Pseudo code below gives a better idea on how the algorithm is running and its implementation details.

Algorithm 2: Mixed radix FFT Graph Flow using conflict free in place policy

input : Time domain inputs n after n dimensional transformation

output: Frequency domain output k after n dimensional transformation

```

1 Initialization parameters;
2 for  $n_1 \leftarrow 0$  to  $N_1 - 1$  do
3   for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
4     ..... // (no. of for loops depend on the no. of stages)
5     Address(Address transformation,  $(n_1 + n_2 + n_3 + \dots + n_n \bmod N_{max}) = x($ 
6        $c_1 n_1 + c_2 n_2 + \dots + c_n n_n \bmod N)$  // conflict free Memory mapping
7   end
8 end
9 if current stage ==  $n^{th}$  then // Set bounds in each stage
10   $b_1 = N_1$ 
11   $b_2 = N_3$ 
12  .....
13   $b_n = N_n$ 
14 end
15 for  $n_1 \leftarrow 0$  to  $N_1 - 1$  do
16   for  $n_2 \leftarrow 0$  to  $N_2 - 1$  do
17     ..... // (no. of for loops depend on the no. of stages)
18     if current stage == radix  $r$  then // Calculations vary for each stage
19       Radix values  $\leftarrow$  Radix- $r$  calculations;
20       Twiddle Factors  $\leftarrow$  Twiddle factors multiplied;
21     end
22   end
23 end
foreach element  $e$  for all the stages  $i$  do Same Algorithm but for respective radix unit;
```

8 Hardware Architecture for mixed radix FFT using PFA

Hardware architecture of mixed radix scheme is shown in the figure 25, which uses combination of Prime factor Algorithm (PFA) and Chinese remainder theorem (CRT) to transform single dimensional index to multiple dimensions. Rather than having separate radix generation unit for various FFTs, we can go for a single radix generation unit that takes care of all FFTs. Architecture that has been shown below depicts all improved characteristics and a complete hardware model to generate mixed radix FFT.

The advantage of hardware model is the improved algorithmic efficiency and reduced number of computations. We are able to simultaneously read all elements necessary to be processed in the radix unit without any memory conflicts, so we are saving many clock cycles and the latency involved in them. The bank number is generated through simple *adder* and *moduli* unit and these consume less hardware.

8.1 Single Port Memory for Multiple Banks

After distributing all elements into different banks, we need at least three cycles to read data, process it and write them back into the same data banks and location. As shown in the figure 26, while processing elements in the radix unit, we can start reading elements from the data banks in

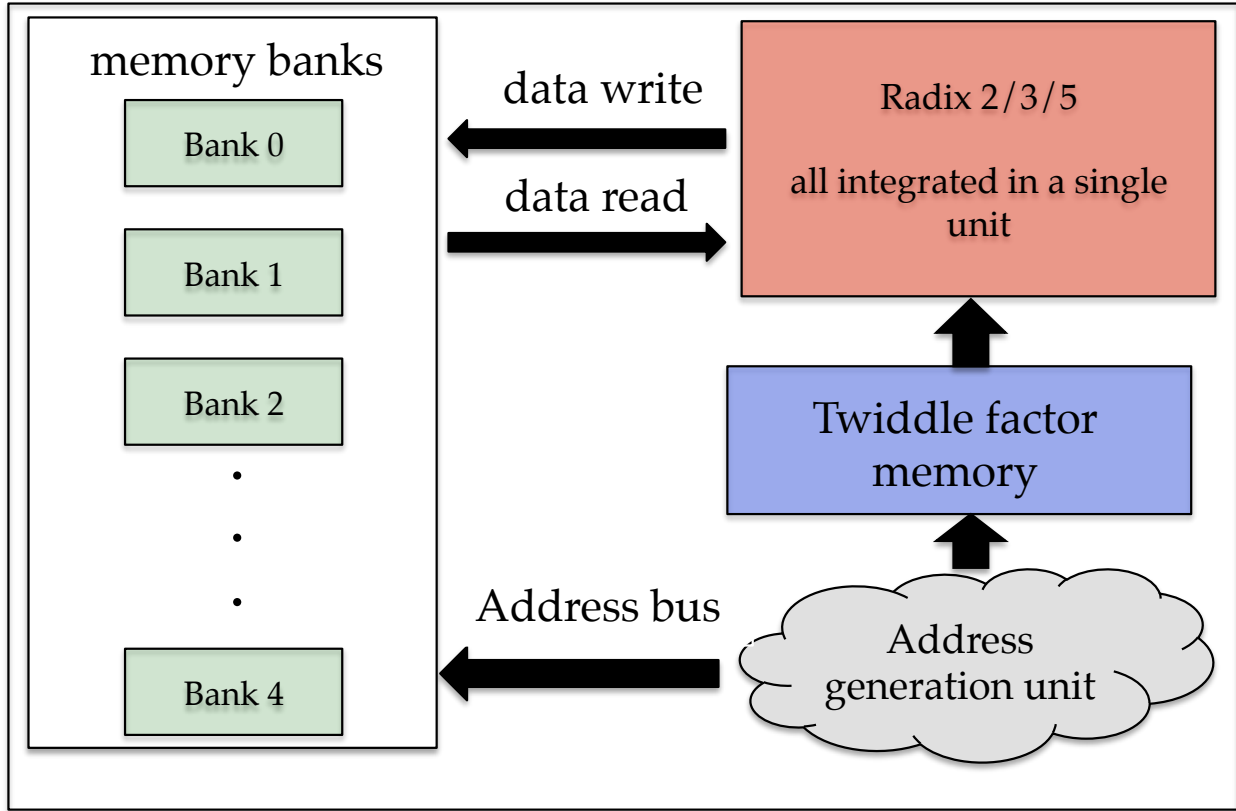


Figure 25: Mixed radix FFT hardware model

the same cycle. At a time, we will be able to process only two data items in 4 clock cycles. Since all the memory banks have single ports, we will not be able to write into the data bank at the same time while reading. Eventually, we won't be able to READ, PROCESS, WRITE in the same cycle, reducing overall throughput of operation. There is a necessity to come-up with hardware architecture that is capable of reading, processing and writing data elements at the same time.

8.2 Improved Hardware Architecture for throughput

We can improve throughput by using duplicate banking scheme as shown in the ?? Modified pipeline diagram is clearly shown in figure 27. As it can be seen, in every cycle we can READ, PROCESS and WRITE. While writing into one series of banks, we are reading from the other se-

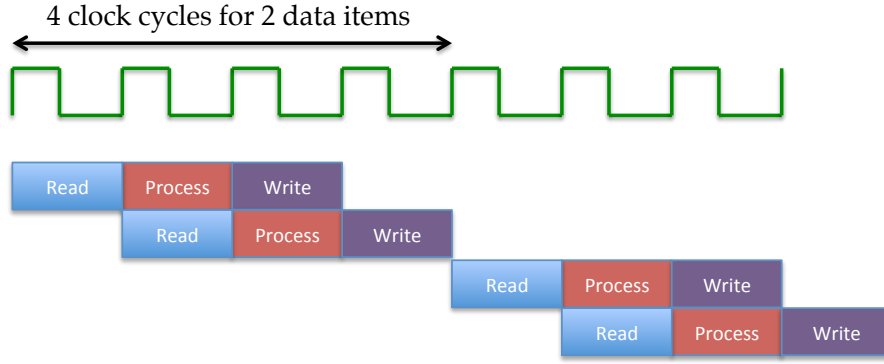


Figure 26: Issues with single port memory banks

ries of banks. In this way we are ensuring to switch between different banks, which results in improved throughput. In a single clock, we are reading, processing data sample and writing it, so the effectively speed up compared to previous method is $2 \times X$

Figure 29 shows the complete solution for $N = 6$ points with Duplicate banking scheme with a duplicate banking scheme shown in different colour. Consider a block length of $N = 6$, radix-3 units are used in the first stage and radix-2 units are used in the second stage. In the first stage we need only two cycles to process all the six elements and in the second stage we need only three clock cycles to process all the elements as shown in the figure below.

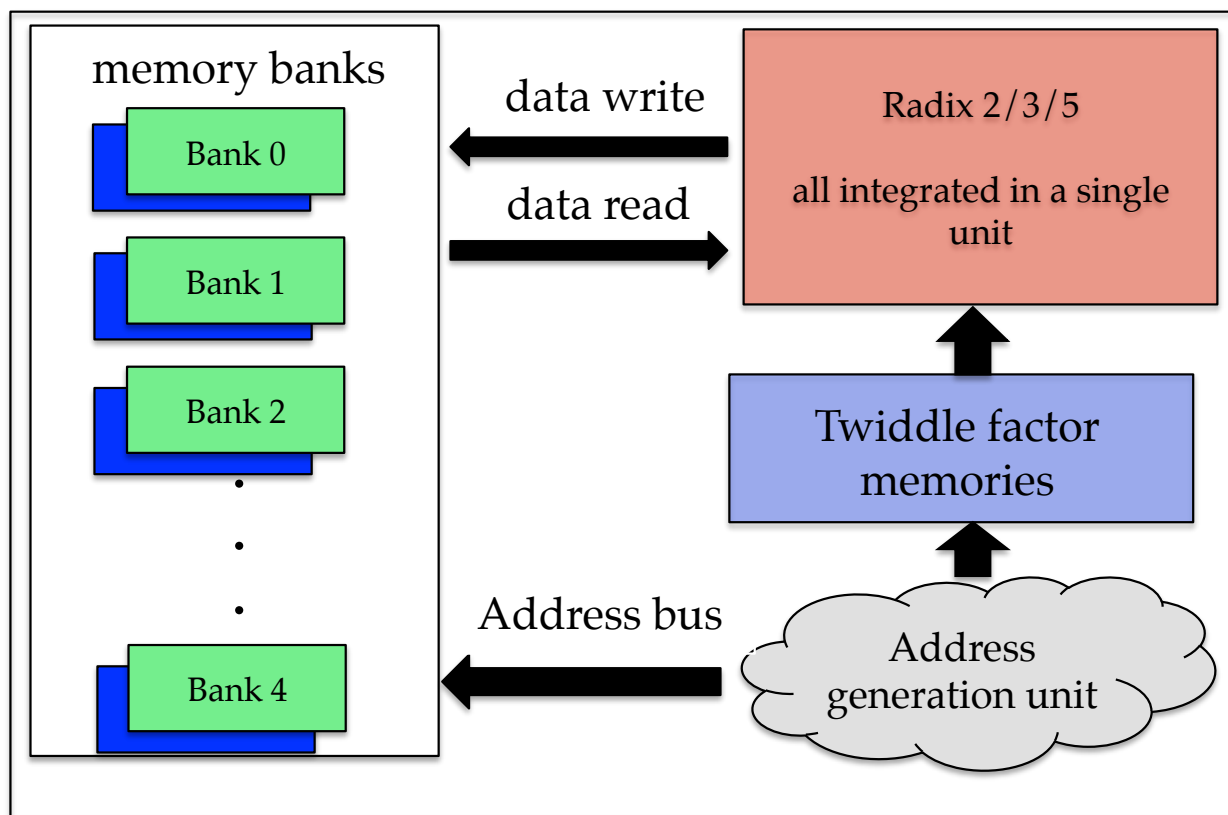


Figure 27: Modified Hardware model using Duplicate Banking Scheme

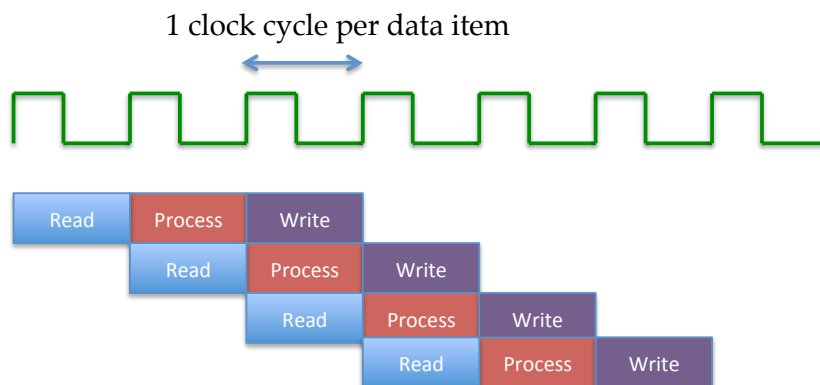
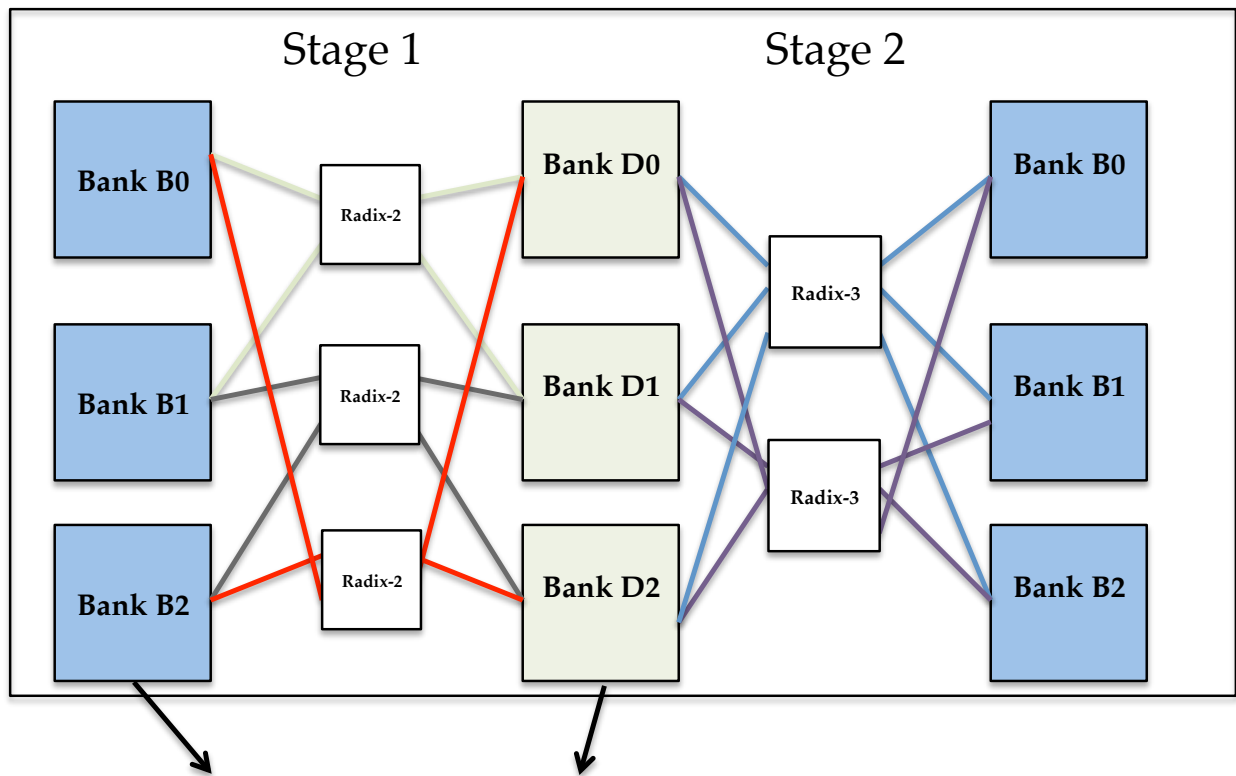


Figure 28: Pipeline diagram for Duplicate Banking scheme



Duplicate Bank - Ping Pong memory

Figure 29: A complete solution for 6 point

9 Speedup comparison

The table shown in fig 31 clearly provides the speed up information for various Block lengths in LTE applications. All block sizes in LTE applications and their speed up information is shown below. Comparison metric is in terms of memory read cycles without Banking scheme and with Banking scheme. Ping-pong memory scheme is also considered along with proposed implementation, switching back and forth between memory banks improves efficiency as discussed earlier. Using Prime Factor Algorithm, in i^{th} stage, if radix- N_i unit is used, we need $\frac{N}{N_i}$ clock cycles for computations.

On the other hand, we need $N \times (no.ofstages)$ if we don't use a mapping scheme. It can be seen that if number of stages increase, number of clock cycles in calculation the FFT also increases. Consider FFT with Prime factor Algorithm, in which if radix unit size increases, then the number of clock cycles needed to compute FFT decrease, which eventually reduces latency. The goal of multi-bank scheme is to intelligently map data points into various banks, in order to reduce the latency.

Speedup table clearly shows that the throughput has improved by $2 \times X$. Similar result is shown in the bar-graph table shown in the fig 31. It can also be seen that whenever the block size is increasing, the algorithm's effective is also increasing. In this way, a chunk of data can be sent easily with a lower latency. Comparison can be made from the previous mixed radix scheme to the new mixed radix scheme

9 Speedup comparison

Serial No.	N(Block length)	M (Radix-2)	P(Radix-3)	Q(Radix-5)	Memory read cycles (OLD)	Memory read cycles (NEW)
1	12	2	1		36	16
2	24	3	1		96	44
3	36	2	2		144	600
4	48	4	1		240	112
5	60	2	1	1	240	92
6	72	3	2		360	156
7	96	5	1		576	272
8	108	2	3		540	216
9	120	3	1	1	600	244
10	144	4	2		864	364
11	192	6	1		1344	640
12	216	3	3		1296	558
13	240	4	1	1	1680	608
14	288	5	2		2016	912
15	972	2	5		6489	2592
16	1152	7	2		10368	4800
15	1296	4	4		10368	4320

Figure 30: Speedup table for various bank lengths

9 Speedup comparison

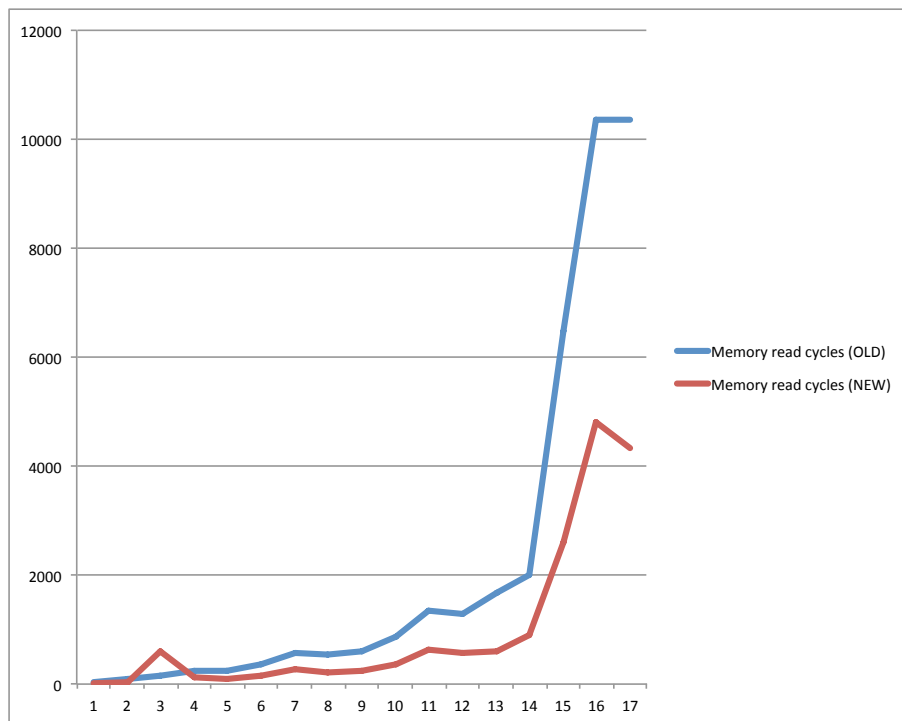


Figure 31: Comparison of OLD and NEW mixed radix scheme

10 Conclusion and Future Work

The idea of banking has introduced a hardware friendly version of mixed radix FFT model. Allocating data elements into different banks in a shuffled manner will do the entire trick here. In this way, we are trying to exploit the inherent parallelism to improve the throughput of our mixed radix scheme. Modifying the algorithm to make it hardware friendly is one of the critical improvements that have been suggested.

More than a speedup of $2 \times x$ was achieved using the scheme. A combination of memory banking, conflict free addressing and Ping-Pong buffers has been used to improve the throughput. One of the biggest advantages is the bank generation hardware which is a simple *adder* and *moduli* units together. For different block sizes, we need pre-computations to allocate data samples to various banks and to decide input and output ordering. An important thing to be remembered is, all pre-computations for various block sizes have to be done much before the hardware model is constructed.

Instead of using a dual port memory, we are duplicating the series of single port memory banks and alternating between them in each stage. Dual port memory is costlier and consumes more space than two single port memories. So, the idea of using duplicate single port memories is better than having a dual port memory.

Bibliography

- [1] Lawrence R Rabiner, Bernard Gold *Theory and Application of Digital Signal Processing* 21.479. 1974.
- [2] L. R. Rabiner and C. M. Rader, eds., *Digital Signal Processing, selected reprints*. New York: IEEE Press, 1972.
- [3] J. H. McClellan and C. M. Rader, *Number Theory in Digital Signal Processing*. Englewood Cliffs, NJ: Prentice-Hall, Inc., 1979.
- [4] C. S. Burrus and T. W. Parks, *DFT/FFT and Convolution Algorithms*. New York: John Wiley and Sons, 1985.
- [5] H. V. Sorensen, D. L. Jones, M. T. Heideman, and C. S. Burrus, *Real valued fast Fourier transform algorithms* IEEE Transactions on Acoustics, Speech, and Signal Processing, vol. 35, June 1987. also in IEEE Press FFT Reprints, by P. Duhamel, 1995.
- [6] R. Stasinski, *The techniques of the generalized fast Fourier transform algorithm* IEEE Transactions on Signal Processing, vol. 39, pp. 1058-1069, May 1991.
- [7] Chen-Fong Hsiao, Yuan Chen and Chen-Yi Lee, *A Generalized Mixed-Radix Algorithm for Memory-Based FFT Processors* IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS?II: EXPRESS BRIEFS, VOL. 57, NO. 1, JANUARY 2010
- [8] R. Meyer and K. Schwarz, *FFT implementation on DSP-chips* Sept. 18 1990. preprint.

Bibliography

- [9] J. Johnson, R. W. Johnson, D. Rodriguez, and R. Tolimieri, *A methodology for designing, modifying, and implementing Fourier transform algorithms on various architectures* Circuits, Systems and Signal Processing, vol. 9, no. 4, pp. 449-500, 1990.
- [10] Jienan Chen, Jianhao Hu, Shuyang Lee and Gerald E. Sobelman *Hardware Efficient Mixed Radix-25/16/9 FFT for LTE Systems* IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS
- [11] J. W. Cooley and J. W. Tukey, *An algorithm for the machine calculation of complex Fourier series* Math. Comput., vol. 19, no. 90, pp. 297-301, 1965.
- [12] L. G. Johnson, *Conflict free memory addressing for dedicated FFT hardware* IEEE Trans. Circuits Syst. II, Analog Digit. Signal Process., vol. 39, no. 5, pp. 312-316, May 1992.
- [13] C. Cheng and K. K. Parhi, *Low-cost fast VLSI algorithm for discrete Fourier transform* IEEE Trans. Circuits Syst. I, Reg. Papers, vol. 54, no. 4, pp. 791-806, Apr. 2007.
- [14] Byung G. Jo and Myung H. Sunwoo, *New Continuous-Flow Mixed-Radix (CFMR) FFT Processor Using Novel In-Place Strategy* IEEE TRANSACTIONS ON CIRCUIT AND SYSTEMS-I: REGULAR PAPERS, VOL. 52, NO. 5, MAY 2005
- [15] Jienan Chen Student Member , Jianhao Hu Member, and Shuyang Li *High Throughput and Hardware Efficient FFT Architecture for LTE Application* 2012 IEEE Wireless Communications and Networking Conference: PHY and Fundamentals
- [16] Song-Nien Tang, Fu-Chiang Jan, Hui-Wen Cheng, Ching-Kai Lin, and Guo-Zua Wu *Multi-mode Memory-Based FFT Processor for Wireless Display FD-OCT Medical Systems* IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS-I: REGULAR PAPERS, VOL. 61, NO. 12, DECEMBER 2014
- [17] C Signey Burrus, *Index Mappings for Multidimensional Formulation of the DFT and Convolution*
- [18] IP LogiCORE Discrete Fourier Transform v3.1 Xilinx
- [19] Freescale Semiconductor Software Optimization of DFTs and IDFTs Using the StarCore SC3850 DSP Core

Bibliography

[20] http://rosettacode.org/wiki/Chinese_remainder_theorem

[21] <http://www.digitechssolutions.com/dsp>